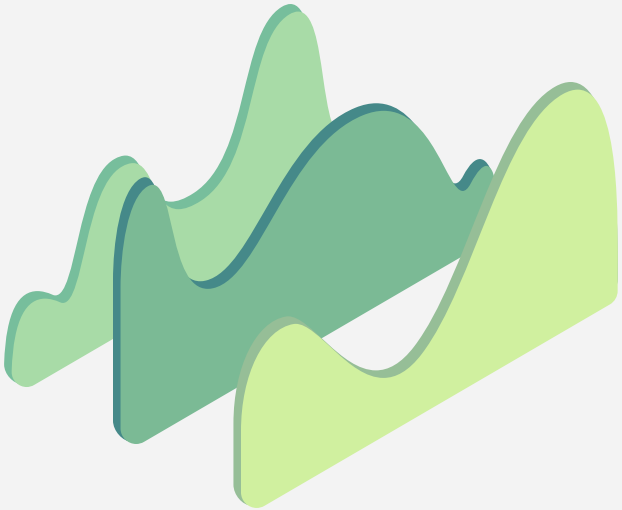
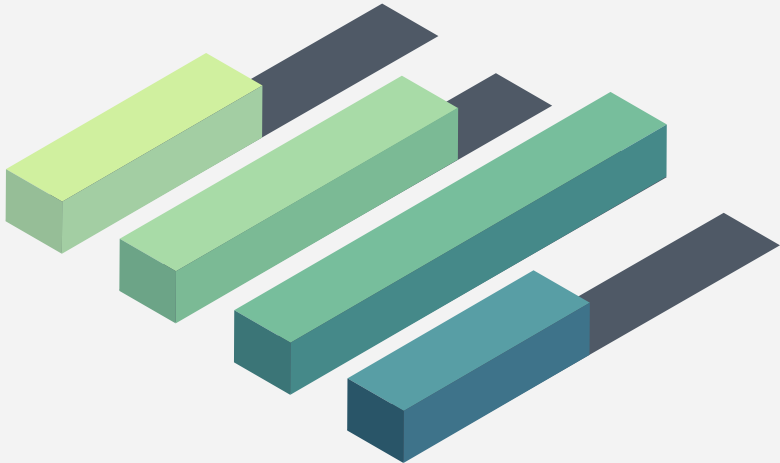
200Lab Education

Basic String Functions

Van Anh - Data Mentor





Definitions

+ A string function is a function that takes a string value as an input regardless of the data type of the returned value. In SQL, there are many built-in string functions that can be used by developers.

+ Some String Basic Functions:
  - UPPER
  - LOWER
  - LENGTH
  - TRIM
  - …



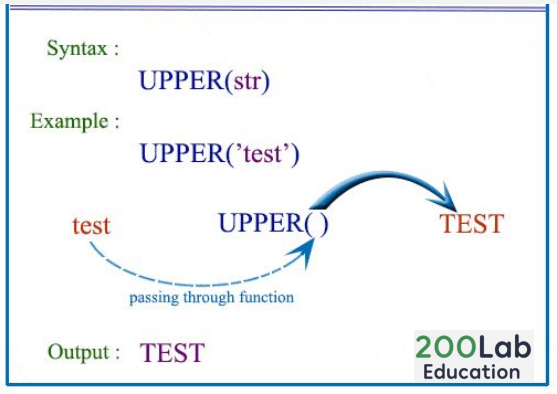
UPPER() Function

+ **Define:** The SQL ***UPPER()*** function is used to convert all characters of a string to uppercase.

+ **Syntax:**

```
UPPER(string)
```

+ **Control Flow:**



UPPER() Function Example

```sql
SELECT
    staff_id,
    UPPER(first_name) AS first_name,
    UPPER(last_name) AS last_name,
    address_id,
    picture,
    email,
    store_id,
    active,
    username,
    `password`,
    last_update
FROM sakila.staff;
```

staff_id	first_name	last_name	address_id	picture	email	store_id	active	
1	MIKE	HILLYER	3	PNG IHDR y u åZÙ... [36365]	Mike.Hillyer@sakilastaff.com	1	1	M
2	JON	STEPHENS	4	[NULL]	Jon.Stephens@sakilastaff.com	2	1	J

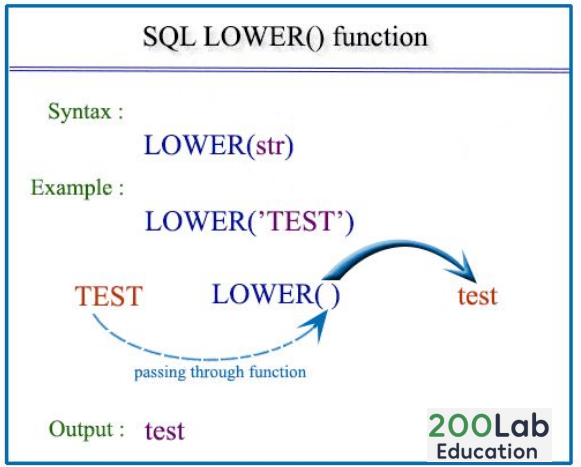
LOWER() Function

+ **Define:** The SQL ***LOWER()*** function is used to convert all characters of a string to lowercase.

+ **Syntax:**

```
LOWER(string)
```

+ **Control Flow:**



LOWER() Function Example

```sql
SELECT
    staff_id,
    UPPER(first_name) AS first_name,
    UPPER(last_name) AS last_name,
    address_id,
    picture,
    LOWER(email) AS email,
    store_id,
    active,
    username,
    `password`,
    last_update
FROM sakila.staff;
```

staff_id	first_name	last_name	address_id	picture	email	store_id	active	username
1	MIKE	HILLYER	3	PNG IHDR y u åZÙ... [36365]	mike.hillyer@sakilastaff.com	1	1	Mike
2	JON	STEPHENS	4	[NULL]	jon.stephens@sakilastaff.com	2	1	Jon

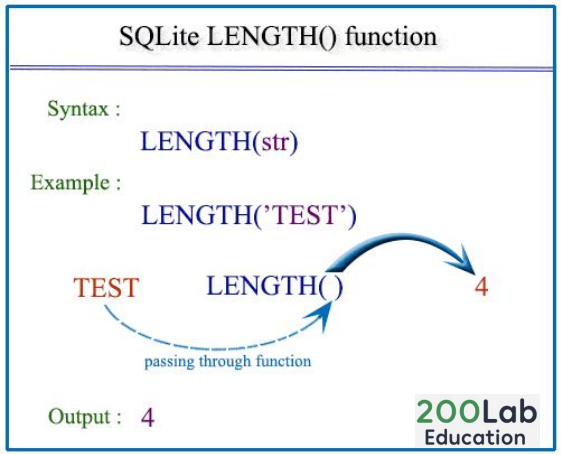
LENGTH() Function

+ **Define:** The SQL ***LENGTH()*** function is returns the length of a string (in bytes).

+ **Syntax:**

```
LENGTH (str)
```

+ **Control Flow:**



LENGTH() Function Example

```sql
SELECT
    film_id,
    title,
    LENGTH(title) AS title_length
FROM sakila.film
ORDER BY 3 DESC;
```

film_id	title	title_length
35	ARACHNOPHOBIA ROLLERCOASTER	27
763	SATISFACTION CONFIDENTIAL	25
224	DESPERATE TRAINSPOTTING	23
297	EXTRAORDINARY CONQUERER	23
905	TRAINSPOTTING STRANGERS	23
287	ENTRAPMENT SATISFACTION	23
880	TELEMARK HEARTBREAKERS	22
740	ROLLERCOASTER BRINGING	22
727	RESURRECTION SILVERADO	22
964	WATERFRONT DELIVERANCE	22
221	DELIVERANCE MULHOLLAND	22
466	INTOLERABLE INTENTIONS	22
174	CONFIDENTIAL INTERVIEW	22
178	CONNECTION MICROCOSMOS	22
366	GOLDFINGER SENSIBILITY	22
332	FRANKENSTEIN STRANGER	21

LENGTH() Function Example (Cont.)

```sql
SELECT
    film_id,
    title,
    description,
    LENGTH(description) AS length_description,
    rating
FROM sakila.film
WHERE LENGTH(description) >= 100;
```

film_id	title	description	length_description	rating
704	PURE RUNNER	A Thoughtful Documentary of a Student And a Madman who must Challenge a Squirrel in A Manhattan Penthou	105	NC-17
705	PURPLE MOVIE	A Boring Display of a Pastry Chef And a Sumo Wrestler who must Discover a Frisbee in An Abandoned Amusem	112	R
710	RAGE GAMES	A Fast-Paced Saga of a Astronaut And a Secret Agent who must Escape a Hunter in An Abandoned Amusement	107	R
711	RAGING AIRPLANE	A Astounding Display of a Secret Agent And a Technical Writer who must Escape a Mad Scientist in A Jet Boat	107	R
716	REAP UNFAITHFUL	A Thrilling Epistle of a Composer And a Sumo Wrestler who must Challenge a Mad Cow in A MySQL Convention	104	PG-13
717	REAR TRADING	A Awe-Inspiring Reflection of a Forensic Psychologist And a Secret Agent who must Succumb a Pastry Chef in S	121	NC-17
725	REQUIEM TYCOON	A Unbelieveable Character Study of a Cat And a Database Administrator who must Pursue a Teacher in A Mona	110	R
728	REUNION WITCHES	A Unbelieveable Documentary of a Database Administrator And a Frisbee who must Redeem a Mad Scientist in	121	R
733	RIVER OUTLAW	A Thrilling Character Study of a Squirrel And a Lumberjack who must Face a Hunter in A MySQL Convention	103	PG-13
735	ROBBERS JOON	A Thoughtful Story of a Mad Scientist And a Waitress who must Confront a Forensic Psychologist in Soviet Geo	112	PG-13
738	ROCKETEER MOTHER	A Awe-Inspiring Character Study of a Robot And a Sumo Wrestler who must Discover a Womanizer in A Shark T	108	PG-13
745	ROSES TREASURE	A Astounding Panorama of a Monkey And a Secret Agent who must Defeat a Woman in The First Manned Space	110	PG-13
747	ROXANNE REBEL	A Astounding Story of a Pastry Chef And a Database Administrator who must Fight a Man in The Outback	100	R
749	RULES HUMAN	A Beautiful Epistle of a Astronaut And a Student who must Confront a Monkey in An Abandoned Fun House	101	R
758	SAINTS BRIDE	A Fateful Tale of a Technical Writer And a Composer who must Pursue a Explorer in The Gulf of Mexico	100	G

TRIM() Function

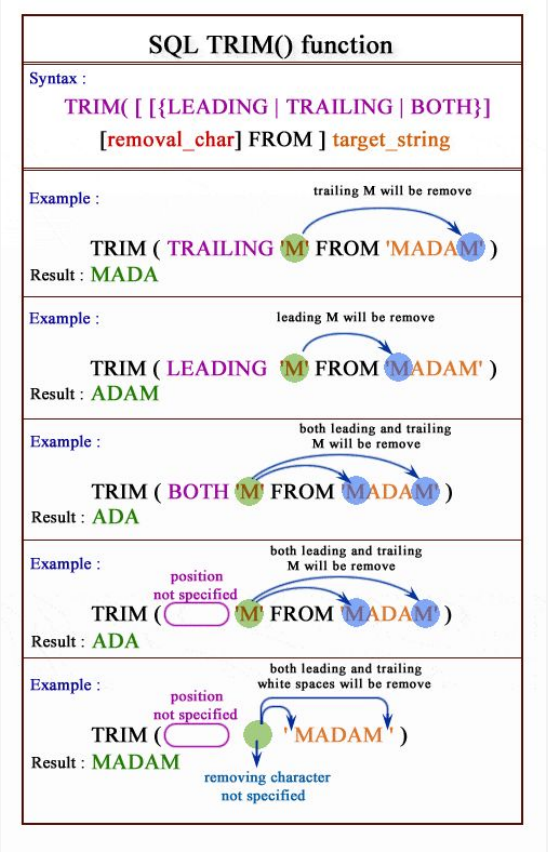
+ **Define:** The SQL ***TRIM()*** removes leading and trailing characters(or both) from a character string

+ **Syntax:**

TRIM White Space	*TRIM Some Characters*
TRIM(String)	**TRIM**(**BOTH** Remove String **FROM** String)
LTRIM(String)	**TRIM**(**LEADING** Remove String **FROM** String)
RTRIM(String)	**TRIM**(**TRAILING** Remove String **FROM** String)

TRIM() Function (Cont.)

+ Control Flow:



TRIM() Function Example

1. String Original

```
SELECT
    "Here is a string" AS str,
    LENGTH("Here is a string") AS len_str
```

ABC str	123 len_str
Here is a string	16

2. TRIM() Handle

```
SELECT
    TRIM(LEADING 'H' FROM 'Here is a string') AS str,
    LENGTH(TRIM(LEADING 'H' FROM 'Here is a string')) AS len_str
```

ABC str	123 len_str
ere is a string	15

TRIM() Function Example

2. TRIM() Handle:

```
SELECT
    TRIM(TRAILING 'H' FROM 'Here is a stringH') AS str,
    LENGTH(TRIM(TRAILING 'H' FROM 'Here is a stringH')) AS len_str
```

ABC str	123 len_str
Here is a string	16

```
SELECT
    TRIM(BOTH 'H' FROM 'Here is a stringH') AS str,
    LENGTH(TRIM(BOTH 'H' FROM 'Here is a stringH')) AS len_str
```

```
SELECT
    TRIM('H' FROM 'Here is a stringH') AS str,
    LENGTH(TRIM('H' FROM 'Here is a stringH')) AS len_str
```

ABC str	123 len_str
ere is a string	15

TRIM() Function Example

1. String Original

```
SELECT
    "   Here is a string   " AS str,
    LENGTH("   Here is a string   ") AS len_str
```



ABC str	123 len_str
Here is a string	22

2. TRIM() Handle

```
SELECT
    LTRIM("   Here is a string   ") AS str,
    LENGTH(LTRIM("   Here is a string   ")) AS len_str
```



ABC str	123 len_str
Here is a string	19

TRIM() Function Example

2. TRIM() Handle:

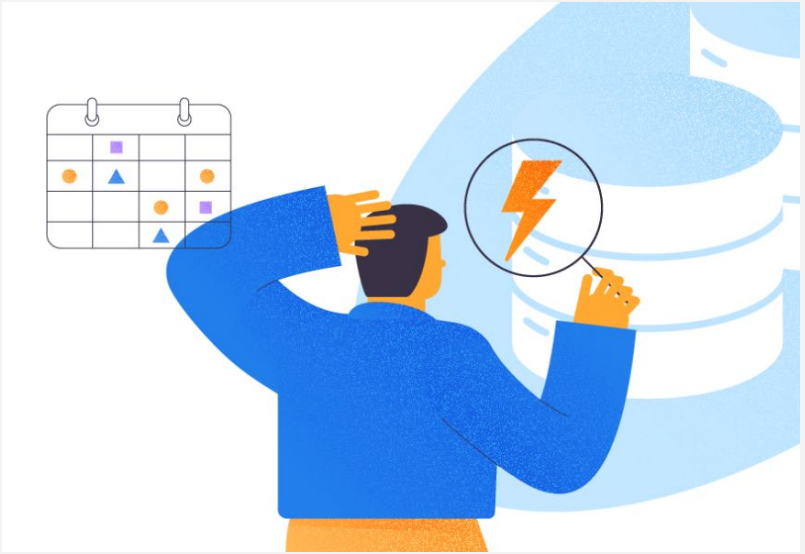
```
SELECT
    RTRIM("   Here is a string   ") AS str,
    LENGTH(RTRIM("   Here is a string   ")) AS len_str
```

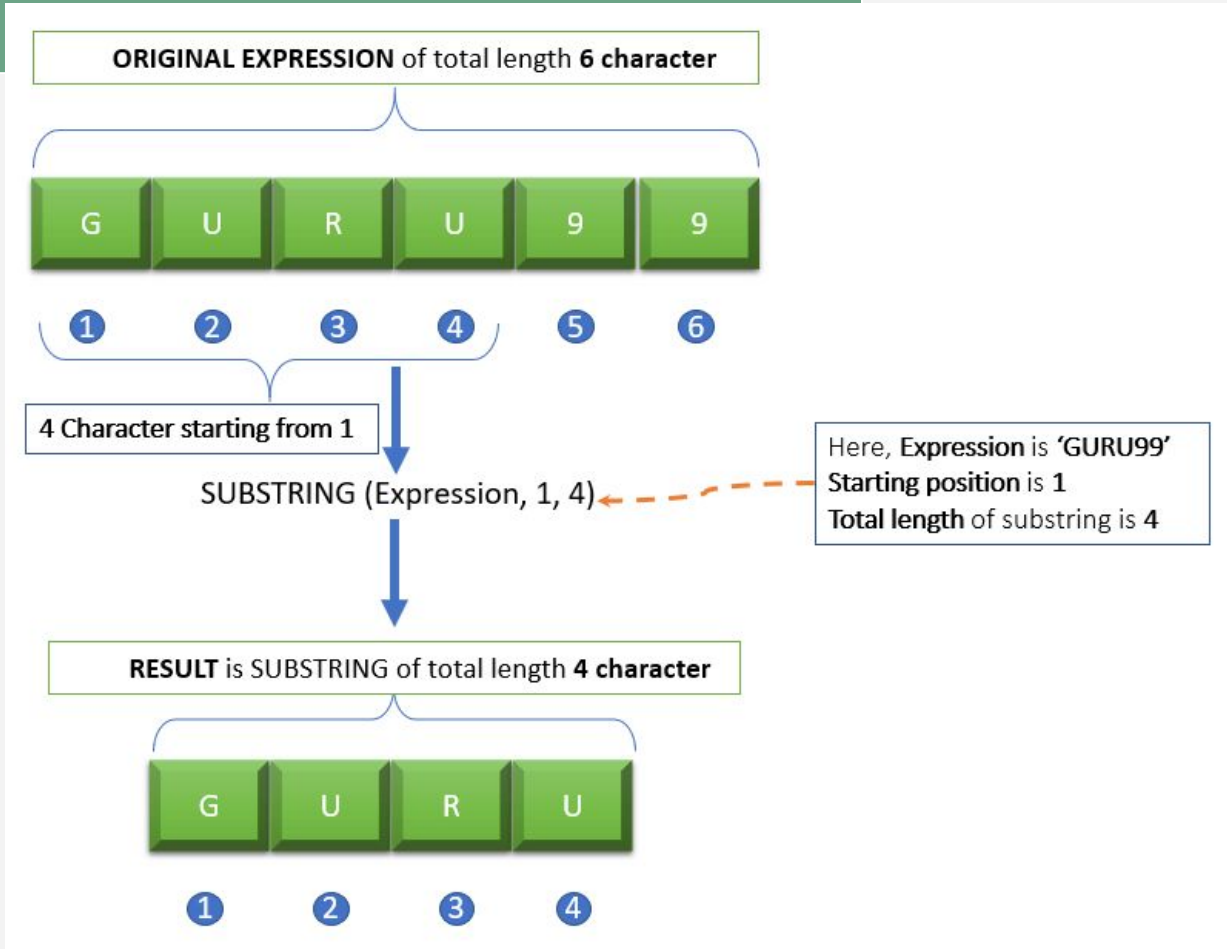
ABC str	123 len_str
Here is a string	19

```
SELECT
    TRIM("   Here is a string   ") AS str,
    LENGTH(TRIM("   Here is a string   ")) AS len_str
```

ABC str	123 len_str
Here is a string	16


f(x)

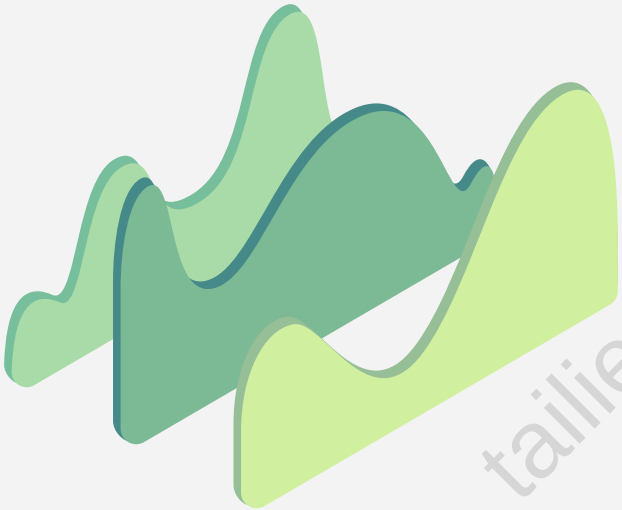


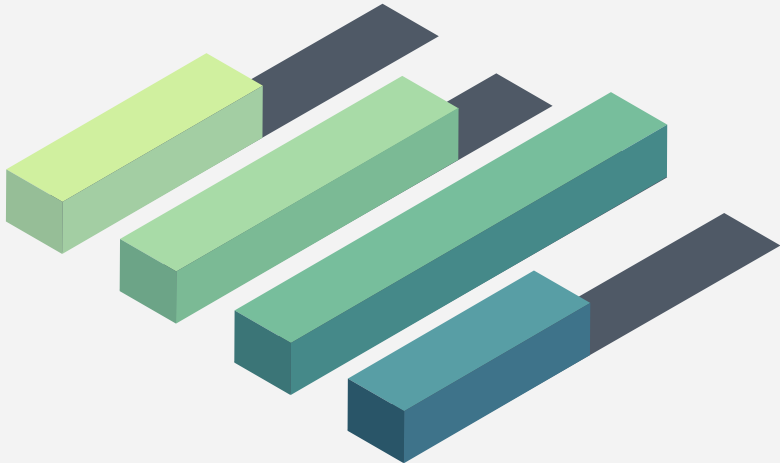

ORIGINAL EXPRESSION of total length 6 character
G
U
R
U
9
9
1
2
3
4
5
6
4 Character starting from 1
SUBSTRING (Expression, 1, 4)
Here, Expression is 'GURU99'
Starting position is 1
Total length of substring is 4
RESULT is SUBSTRING of total length 4 character
G
U
R
U
1
2
3
4

200Lab Education

Advanced String Functions

Van Anh - Data Mentor





tailieusharefree.blogspot.com

Definitions

+ A string function is a function that takes a string value as an input regardless of the data type of the returned value. In SQL Server, there are many built-in string functions that can be used by developers.

+ Some Advanced String Functions:
  - SUBSTRING
  - REPLACE
  - POSITION
  - CONVERT
  - …



tailieusharefree.blogspot.com

SUBSTRING() Function

+ Define: The **SUBSTRING()** function extracts a substring from a string (starting at any position).

+ Syntax:

```
SUBSTRING(string, start, length)
```

OR:

```
SUBSTRING(string FROM start FOR length)
```

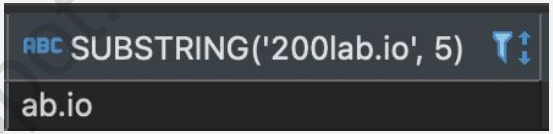
+ Parameter:

Parameter	Description
string	Required. The string to extract from
start	Required. The start position. Can be both a positive or negative number. If it is a positive number, this function extracts from the beginning of the string. If it is a negative number, this function extracts from the end of the string
length	Optional. The number of characters to extract. If omitted, the whole string will be returned (from the *start* position)

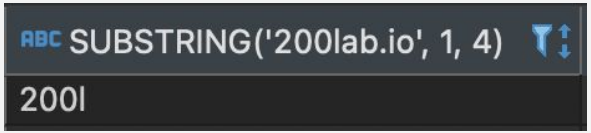
tailieusharefree.blogspot.com

SUBSTRING() Example

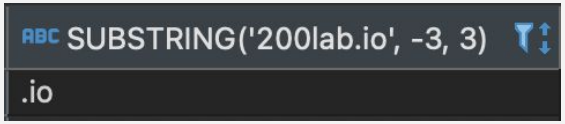
```
SELECT SUBSTRING('200lab.io', 5);
SELECT SUBSTRING('200lab.io' FROM 5);
```



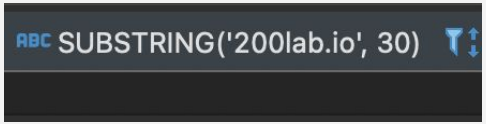

```
SELECT SUBSTRING('200lab.io', 1, 4);
SELECT SUBSTRING('200lab.io' FROM 1 FOR 4);
```




```
SELECT SUBSTRING('200lab.io', -3, 3);
SELECT SUBSTRING('200lab.io' FROM -3 FOR 3);
```




```
SELECT SUBSTRING('200lab.io', 30);
```



ailieuchiretea.blogspot.com

REPLACE() Function

+ Define: The **REPLACE()** function replaces all occurrences of a substring within a string, with a new substring.

+ Syntax:

```
REPLACE(string, from_string, new_string)
```

+ Parameter:

Parameter	Description
string	Required. The original string
from_string	Required. The substring to be replaced
new_string	Required. The new replacement substring

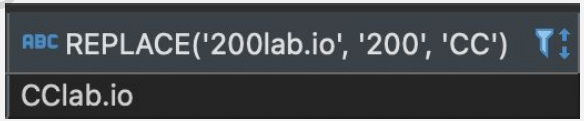
tailieusharefree.blogspot.com

REPLACE() Example

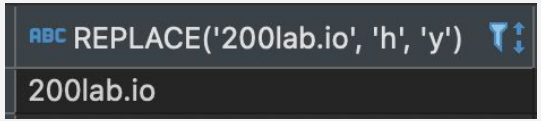
```
SELECT REPLACE('200lab.io', '.io', ' Education');
```



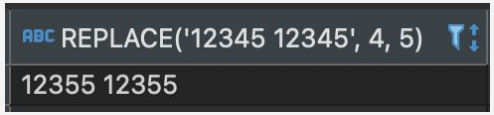

```
SELECT REPLACE('200lab.io', '200', 'CC');
```




```
SELECT REPLACE('200lab.io', 'h', 'y');
```




```
SELECT REPLACE('12345 12345', 4, 5);
```



tailieusharefree.blogspot.com

POSITION() Function

+ Define: The **POSITION()** function returns the position of the first occurrence of a substring in a string.

If the substring is not found within the original string, this function returns 0.

This function performs a case-insensitive search.

+ Syntax:

```
POSITION(substring IN string)
```

+ Parameter:

Parameter	Description
substring	Required. The substring to search for in *string*
string	Required. The original string that will be searched

tailieusharefree.blogspot.com

POSITION() Example

```
SELECT POSITION('E' IN '200lab Education');
```

```
SELECT POSITION('e' IN '200lab Education');
```

123 POSITION('e' IN '200lab Education')
8

```
SELECT POSITION('a' IN '200lab Education');
```

123 POSITION('a' IN '200lab Education')
5

```
SELECT POSITION('lab' IN '200lab Education');
```

123 POSITION('lab' IN '200lab Education')
4

```
SELECT POSITION('f' IN '200lab Education');
```

123 POSITION('f' IN '200lab Education')
0

tailieushare.blogspot.com

POSITION() Example (Cont.)

```
SELECT
    customer_id,
    first_name,
    last_name,
    POSITION('a' IN last_name) AS a_loc
FROM sakila.customer;
```

customer_id	first_name	last_name	a_loc
1	MARY	SMITH	0
2	PATRICIA	JOHNSON	0
3	LINDA	WILLIAMS	6
4	BARBARA	JONES	0
5	ELIZABETH	BROWN	0
6	JENNIFER	DAVIS	2
7	MARIA	MILLER	0
8	SUSAN	WILSON	0
9	MARGARET	MOORE	0
10	DOROTHY	TAYLOR	2
11	LISA	ANDERSON	1
12	NANCY	THOMAS	5
13	KAREN	JACKSON	2
14	BETTY	WHITE	0
15	HELEN	HARRIS	2
16	SANDRA	MARTIN	2
17	DONNA	THOMPSON	0
18	CAROL	GARCIA	2
19	RUTH	MARTINEZ	2

tailieushare.blogspot.com

CONVERT() Function

+ **Define**: The **CONVERT()** function converts a value into the specified datatype or character set.

+ **Syntax:**

```
CONVERT(value, type)
```

OR:

```
CONVERT(value USING charset)
```

tailieusharefree.blogspot.com

CONVERT() Function

+ Parameter:

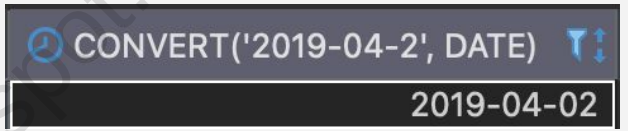
Parameter	Description
value	Required. The value to convert
type	Required. The datatype to convert to. Can be one of the following:
charset	Required. The character set to convert to

Value	Description
DATE	Converts *value* to DATE. Format: "YYYY-MM-DD"
DATETIME	Converts *value* to DATETIME. Format: "YYYY-MM-DD HH:MM:SS"
DECIMAL	Converts *value* to DECIMAL. Use the optional M and D parameters to specify the maximum number of digits (M) and the number of digits following the decimal point (D).
TIME	Converts *value* to TIME. Format: "HH:MM:SS"
CHAR	Converts *value* to CHAR (a fixed length string)
NCHAR	Converts *value* to NCHAR (like CHAR, but produces a string with the national character set)
SIGNED	Converts *value* to SIGNED (a signed 64-bit integer)
UNSIGNED	Converts *value* to UNSIGNED (an unsigned 64-bit integer)
BINARY	Converts *value* to BINARY (a binary string)

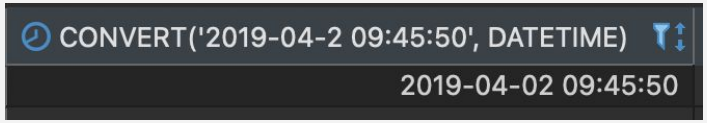
tailieusharefree.blogspot.com

CONVERT() Example

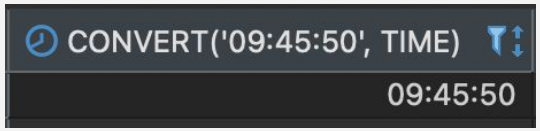
```
SELECT CONVERT('2019-04-2', DATE);
```



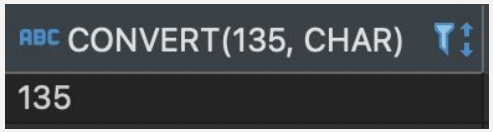

```
SELECT CONVERT('2019-04-2 09:45:50', DATETIME);
```




```
SELECT CONVERT('09:45:50', TIME);
```




```
SELECT CONVERT(135, CHAR);
```



tailieushanfree.blogspot.com

CONVERT() Example (Cont.)

```sql
SELECT
    payment_id,
    customer_id,
    staff_id,
    CONVERT(rental_id, CHAR) AS rental_id,
    amount,
    payment_date,
    last_update
FROM sakila.payment;
```

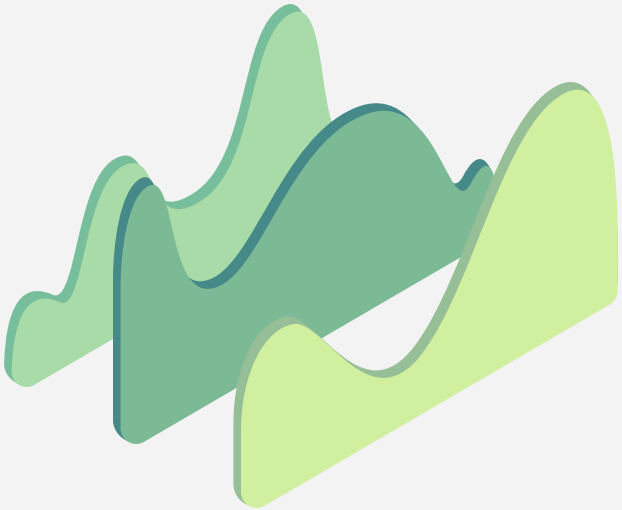
payment_id	customer_id	staff_id	rental_id	amount	payment_date	last_update
1	1	1	76	2.99	2005-05-25 11:30:37	2006-02-15 22:12:30
2	1	1	573	0.99	2005-05-28 10:35:23	2006-02-15 22:12:30
3	1	1	1185	5.99	2005-06-15 00:54:12	2006-02-15 22:12:30
4	1	2	1422	0.99	2005-06-15 18:02:53	2006-02-15 22:12:30
5	1	2	1476	9.99	2005-06-15 21:08:46	2006-02-15 22:12:30
6	1	1	1725	4.99	2005-06-16 15:18:57	2006-02-15 22:12:30
7	1	1	2308	4.99	2005-06-18 08:41:48	2006-02-15 22:12:30
8	1	2	2363	0.99	2005-06-18 13:33:59	2006-02-15 22:12:30
9	1	1	3284	3.99	2005-06-21 06:24:45	2006-02-15 22:12:30
10	1	2	4526	5.99	2005-07-08 03:17:05	2006-02-15 22:12:30
11	1	1	4611	5.99	2005-07-08 07:33:56	2006-02-15 22:12:30
12	1	1	5244	4.99	2005-07-09 13:24:07	2006-02-15 22:12:30
13	1	1	5326	4.99	2005-07-09 16:38:01	2006-02-15 22:12:30
14	1	1	6163	7.99	2005-07-11 10:13:46	2006-02-15 22:12:30
15	1	2	7273	2.99	2005-07-27 11:31:22	2006-02-15 22:12:30
16	1	1	7841	4.99	2005-07-28 09:04:45	2006-02-15 22:12:30

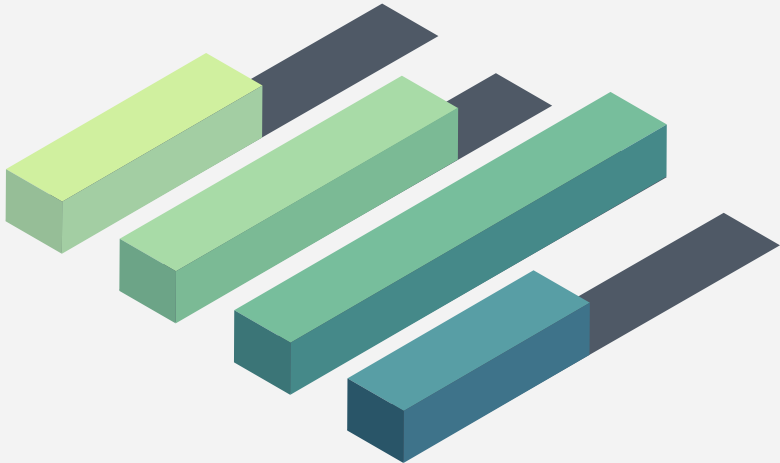
taimeuskarate.blogspot.com

200Lab Education

Date Functions In SQL

Van Anh - Data Mentor





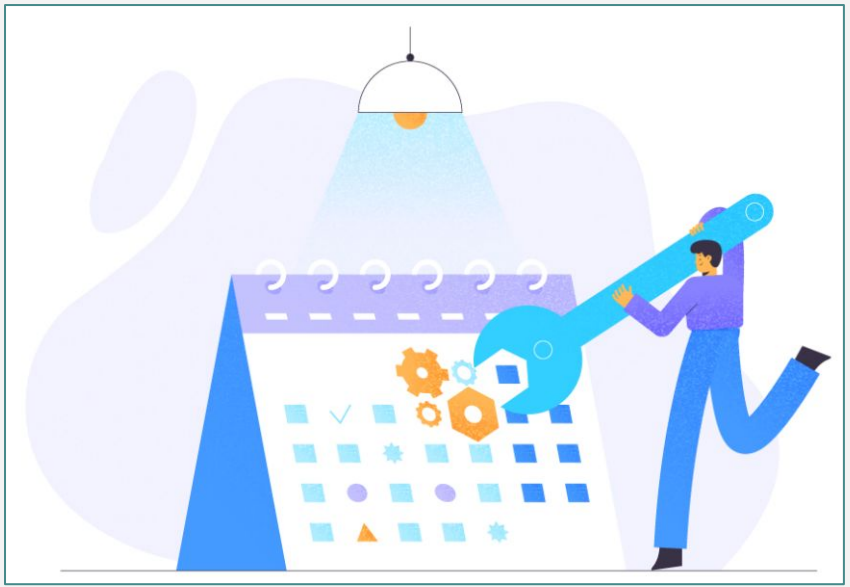
Definitions

+ This lesson introduces techniques for performing simple date arithmetic. Recipes cover common tasks such as adding days to dates, finding the number of business days between dates, and finding the difference between dates in days.

+ Being able to successfully manipulate dates with your RDBMS's built-in functions can greatly improve your productivity.

+ Some Date Functions:

- DATE_ADD
- DATE_SUB
- CURRENT_DATE
- DATEDIFF
- EXTRACT
- DATE_FORMAT
- STR_TO_DATE
- …



DATE_ADD() Function

+ Define: The **DATE_ADD()** function adds a time/date interval to a date and then returns the date.

+ Syntax:

```
DATE_ADD(date, INTERVAL value addunit)
```

+ Parameter:

Parameter	Description
date	Required. The date to be modified
value	Required. The value of the time/date interval to add. Both positive and negative values are allowed
addunit	Required. The type of interval to add.

DATE_ADD() Function

+ Parameter:

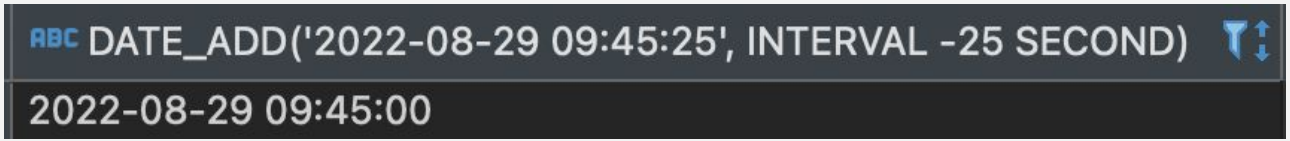
addunit	Required. The type of interval to add. Can be one of the following values: • MICROSECOND • SECOND • MINUTE • HOUR • DAY • WEEK • MONTH • QUARTER • YEAR • SECOND_MICROSECOND • MINUTE_MICROSECOND • MINUTE_SECOND • HOUR_MICROSECOND • HOUR_SECOND • HOUR_MINUTE • DAY_MICROSECOND • DAY_SECOND • DAY_MINUTE • DAY_HOUR • YEAR_MONTH

DATE_ADD() Example

```
SELECT DATE_ADD('2022-08-29 09:45:25.000005', INTERVAL 5 MICROSECOND);
```




```
SELECT DATE_ADD('2022-08-29 09:45:25', INTERVAL -25 SECOND);
```



DATE_ADD() Example (Cont.)

```
SELECT DATE_ADD('2022-08-29 09:45:25', INTERVAL 20 MINUTE);
```

DATE_ADD('2022-08-29 09:45:25', INTERVAL 20 MINUTE)
2022-08-29 10:05:25

```
SELECT DATE_ADD('2022-08-29 09:45:25', INTERVAL -5 HOUR);
```

DATE_ADD('2022-08-29 09:45:25', INTERVAL -5 HOUR)
2022-08-29 04:45:25

DATE_ADD() Example (Cont.)

```
SELECT DATE_ADD('2022-08-29', INTERVAL 11 DAY);
```

DATE_ADD('2022-08-29', INTERVAL 11 DAY)
2022-09-09

```
SELECT DATE_ADD('2022-08-29', INTERVAL 15 WEEK);
```

DATE_ADD('2022-08-29', INTERVAL 15 WEEK)
2022-12-12

DATE_ADD() Example (Cont.)

```
SELECT DATE_ADD('2022-08-29', INTERVAL -7 MONTH);
```

DATE_ADD('2022-08-29', INTERVAL -7 MONTH)
2022-01-29

```
SELECT DATE_ADD('2022-08-29', INTERVAL 5 QUARTER);
```

DATE_ADD('2022-08-29', INTERVAL 5 QUARTER)
2023-11-29

```
SELECT DATE_ADD('2022-08-29', INTERVAL 2 YEAR);
```

DATE_ADD('2022-08-29', INTERVAL 2 YEAR)
2024-08-29

DATE_ADD() Example (Cont.)

```sql
SELECT
    payment_id,
    rental_id,
    amount,
    payment_date,
    last_update
FROM sakila.payment
WHERE payment_date >= DATE_ADD('2006-01-01 00:00:00', INTERVAL -1 MONTH)
AND amount > 3;
```

payment_id	rental_id	amount	payment_date	last_update
253	15,813	4.99	2006-02-14 15:16:03	2006-02-15 22:12:31
385	13,780	4.99	2006-02-14 15:16:03	2006-02-15 22:12:32
416	13,798	3.98	2006-02-14 15:16:03	2006-02-15 22:12:32
600	12,222	4.99	2006-02-14 15:16:03	2006-02-15 22:12:33
1,177	13,351	5.98	2006-02-14 15:16:03	2006-02-15 22:12:39
1,201	15,644	3.98	2006-02-14 15:16:03	2006-02-15 22:12:39
1,228	13,428	4.99	2006-02-14 15:16:03	2006-02-15 22:12:39
1,453	12,001	4.99	2006-02-14 15:16:03	2006-02-15 22:12:42
1,482	11,657	7.98	2006-02-14 15:16:03	2006-02-15 22:12:42
1,564	15,714	4.99	2006-02-14 15:16:03	2006-02-15 22:12:43
1,670	12,489	9.98	2006-02-14 15:16:03	2006-02-15 22:12:45
1,766	13,333	4.99	2006-02-14 15:16:03	2006-02-15 22:12:46
2,059	13,534	8.97	2006-02-14 15:16:03	2006-02-15 22:12:50
2,268	11,563	4.99	2006-02-14 15:16:03	2006-02-15 22:12:53
2,387	12,719	4.99	2006-02-14 15:16:03	2006-02-15 22:12:54
2,503	12,902	4.99	2006-02-14 15:16:03	2006-02-15 22:12:56
2,575	15,371	4.99	2006-02-14 15:16:03	2006-02-15 22:12:58
2,932	15,294	4.99	2006-02-14 15:16:03	2006-02-15 22:13:04
3,090	12,506	4.99	2006-02-14 15:16:03	2006-02-15 22:13:07

DATE_SUB() Function

+ Definition: The **DATE_SUB()** function subtracts a time/date interval from a date and then returns the date.

+ Syntax:

```
DATE_SUB(date, INTERVAL value interval)
```

+ Parameter:

Parameter	Description
date	Required. The date to be modified
value	Required. The value of the time/date interval to subtract. Both positive and negative values are allowed
interval	Required. The type of interval to subtract.

DATE_SUB() Function

+ Parameter:

interval	Required. The type of interval to subtract. Can be one of the following values:

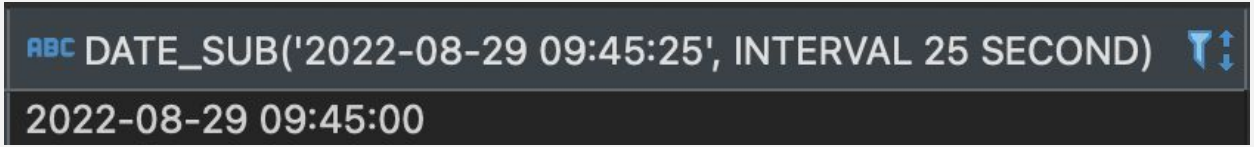
- MICROSECOND
- SECOND
- MINUTE
- HOUR
- DAY
- WEEK
- MONTH
- QUARTER
- YEAR
- SECOND_MICROSECOND
- MINUTE_MICROSECOND
- MINUTE_SECOND
- HOUR_MICROSECOND
- HOUR_SECOND
- HOUR_MINUTE
- DAY_MICROSECOND
- DAY_SECOND
- DAY_MINUTE
- DAY_HOUR
- YEAR_MONTH

DATE_SUB() Example

```
SELECT DATE_SUB('2022-08-29 09:45:25.000001', INTERVAL 5 MICROSECOND);
```




```
SELECT DATE_SUB('2022-08-29 09:45:25', INTERVAL 25 SECOND);
```



DATE_SUB() Example (Cont.)

```
SELECT DATE_SUB('2022-08-29 09:45:25', INTERVAL 40 MINUTE);
```

DATE_SUB('2022-08-29 09:45:25', INTERVAL 40 MINUTE)
2022-08-29 09:05:25

```
SELECT DATE_SUB('2022-08-29 09:45:25', INTERVAL 5 HOUR);
```

DATE_SUB('2022-08-29 09:45:25', INTERVAL 5 HOUR)
2022-08-29 04:45:25

DATE_SUB() Example (Cont.)

```
SELECT DATE_SUB('2022-08-29 09:45:25', INTERVAL 12 DAY);
```

DATE_SUB('2022-08-29 09:45:25', INTERVAL 12 DAY)
2022-08-17 09:45:25

```
SELECT DATE_SUB('2022-08-29 09:45:25', INTERVAL 14 WEEK);
```

DATE_SUB('2022-08-29 09:45:25', INTERVAL 14 WEEK)
2022-05-23 09:45:25

DATE_SUB() Example (Cont.)

```
SELECT DATE_SUB('2022-08-29', INTERVAL -2 MONTH);
```

DATE_SUB('2022-08-29', INTERVAL -2 MONTH)
2022-10-29

```
SELECT DATE_SUB('2022-08-29', INTERVAL 8 QUARTER);
```

DATE_SUB('2022-08-29', INTERVAL 8 QUARTER)
2020-08-29

DATE_SUB() Example (Cont.)

```
SELECT DATE_SUB('2022-08-29', INTERVAL 1 YEAR);
```

DATE_SUB('2022-08-29', INTERVAL 1 YEAR)
2021-08-29

```
SELECT DATE_SUB('2022-08-29 09:45:25.000001', INTERVAL '12.000001' SECOND_MICROSECOND);
```

DATE_SUB('2022-08-29 09:45:25.000001', INTERVAL '12.000001' SECOND_MICROSECOND)
2022-08-29 09:45:13

DATE_SUB() Example (Cont.)

```
SELECT DATE_SUB('2022-08-29 09:45:25.000001', INTERVAL '3:12.000001' MINUTE_MICROSECOND);
```

DATE_SUB('2022-08-29 09:45:25.000001', INTERVAL '3:12.000001' MINUTE_MICROSECOND)
2022-08-29 09:42:13

```
SELECT DATE_SUB('2022-08-29 09:45:25', INTERVAL '3:12' MINUTE_SECOND);
```

DATE_SUB('2022-08-29 09:45:25', INTERVAL '3:12' MINUTE_SECOND)
2022-08-29 09:42:13

DATE_SUB() Example (Cont.)

```
SELECT DATE_SUB('2022-08-29 09:45:25.000001', INTERVAL '1:03:12.000001' HOUR_MICROSECOND);
```

DATE_SUB('2022-08-29 09:45:25.000001', INTERVAL '1:03:12.000001' HOUR_MICROSECOND)
2022-08-29 08:42:13

```
SELECT DATE_SUB('2022-08-29 09:45:25', INTERVAL '5:03:12' HOUR_SECOND);
```

DATE_SUB('2022-08-29 09:45:25', INTERVAL '5:03:12' HOUR_SECOND)
2022-08-29 04:42:13

```
SELECT DATE_SUB('2022-08-29 09:45:25', INTERVAL '3:03' HOUR_MINUTE);
```

DATE_SUB('2022-08-29 09:45:25', INTERVAL '3:03' HOUR_MINUTE)
2022-08-29 06:42:25

DATE_SUB() Example (Cont.)

```sql
SELECT
    payment_id,
    rental_id,
    amount,
    payment_date,
    last_update
FROM sakila.payment
WHERE payment_date >= '2006-01-01 00:00:00'
AND payment_date <= DATE_SUB('2006-01-01 00:00:00', INTERVAL -2 MONTH)
AND amount > 3
ORDER BY payment_date ASC;
```

```sql
SELECT
    payment_id,
    rental_id,
    amount,
    payment_date,
    last_update
FROM sakila.payment
WHERE payment_date >= DATE_SUB('2006-03-01 00:00:00', INTERVAL 2 MONTH)
AND payment_date < '2006-03-01 00:00:00'
AND amount > 3
ORDER BY payment_date ASC;
```

payment_id	rental_id	amount	payment_date	last_update
11,886	13,106	4.99	2006-02-14 15:16:03	2006-02-15 22:18:49
6,133	13,374	4.99	2006-02-14 15:16:03	2006-02-15 22:14:27
6,159	12,672	3.98	2006-02-14 15:16:03	2006-02-15 22:14:28
6,621	12,736	4.99	2006-02-14 15:16:03	2006-02-15 22:14:45
6,809	13,756	4.99	2006-02-14 15:16:03	2006-02-15 22:14:51
7,243	12,066	7.98	2006-02-14 15:16:03	2006-02-15 22:15:06
7,302	13,025	3.98	2006-02-14 15:16:03	2006-02-15 22:15:09
7,583	13,538	4.99	2006-02-14 15:16:03	2006-02-15 22:15:20
7,706	12,064	5.98	2006-02-14 15:16:03	2006-02-15 22:15:25
8,166	15,695	4.99	2006-02-14 15:16:03	2006-02-15 22:15:43
9,585	12,759	7.98	2006-02-14 15:16:03	2006-02-15 22:16:47
9,708	15,655	4.99	2006-02-14 15:16:03	2006-02-15 22:16:52
9,772	13,298	3.98	2006-02-14 15:16:03	2006-02-15 22:16:55
9,917	13,421	4.99	2006-02-14 15:16:03	2006-02-15 22:17:03
10,693	13,178	4.99	2006-02-14 15:16:03	2006-02-15 22:17:41
11,148	13,246	4.99	2006-02-14 15:16:03	2006-02-15 22:18:09
5,947	11,577	4.99	2006-02-14 15:16:03	2006-02-15 22:14:19
11,914	14,878	4.99	2006-02-14 15:16:03	2006-02-15 22:18:50
12,112	14,734	3.98	2006-02-14 15:16:03	2006-02-15 22:19:01

CURRENT_DATE () Function

+ Definition: The **CURRENT_DATE()** function returns the current date.

+ Note: The date is returned as "YYYY-MM-DD" (string) or as YYYYMMDD (numeric).

+ Syntax:

```
CURRENT_DATE()
```



CURRENT_DATE() Example

```
SELECT CURRENT_DATE();
```

CURRENT_DATE()
2022-08-30

```
SELECT CURRENT_DATE() + 5;
```

123 CURRENT_DATE() + 5
20,220,835

```
SELECT DATE_SUB(CURRENT_DATE(), INTERVAL 5 DAY);
```

DATE_SUB(CURRENT_DATE(), INTERVAL 5 DAY)
2022-08-25

```
SELECT DATE_ADD(CURRENT_DATE(), INTERVAL 5 DAY);
```

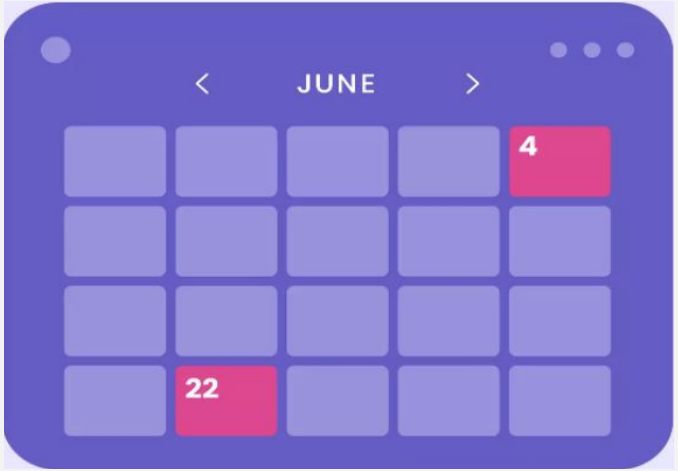
DATE_ADD(CURRENT_DATE(), INTERVAL 5 DAY)
2022-09-04

DATEDIFF() Function

+ Definition: The **DATEDIFF()** function returns the number of days between two date values.

+ Syntax:

```
DATEDIFF(date1, date2)
```



+ Parameter: date1, date2: required. Two dates to calculate the number of days between. (date1 - date2)

DATEDIFF() Example

```
SELECT DATEDIFF('2022-06-22', '2022-06-04');
```

123 DATEDIFF('2022-06-22', '2022-06-04')
18

```
SELECT DATEDIFF(CURRENT_DATE(), '2022-06-04');
```

123 DATEDIFF(CURRENT_DATE() , '2022-06-04')
87

```
SELECT DATEDIFF('2022-08-30 19:50:14', '2022-03-27 19:10:08');
```

123 DATEDIFF('2022-08-30 13:45:14', '2022-03-27')
156

DATEDIFF() Example (Cont.)

```
SELECT DATEDIFF('2022-08-30 13:45:14', '2022-03-27');
```

DATEDIFF('2022-08-30 13:45:14', '2022-03-27')
156

```
SELECT
    *,
    DATEDIFF(last_update, rental_date) AS duration
FROM sakila.rental
ORDER BY duration DESC;
```

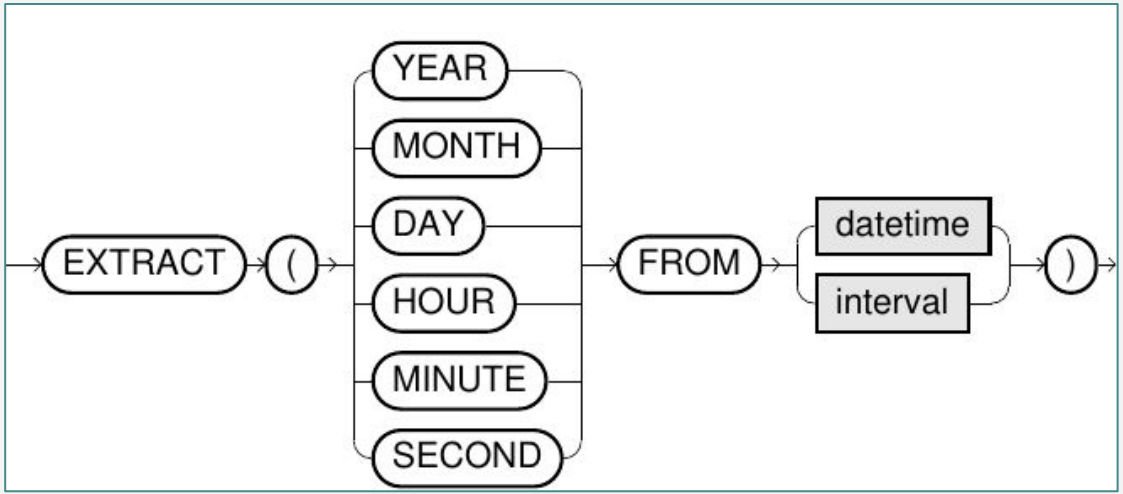
rental_id	rental_date	inventory_id	customer_id	return_date	staff_id	last_update	duration
6	2005-05-24 23:08:07	2,792	549	2005-05-27 01:32:07	1	2006-02-15 21:30:53	267
7	2005-05-24 23:11:53	3,995	269	2005-05-29 20:34:53	2	2006-02-15 21:30:53	267
8	2005-05-24 23:31:46	2,346	239	2005-05-27 23:33:46	2	2006-02-15 21:30:53	267
9	2005-05-25 00:00:40	2,580	126	2005-05-28 00:22:40	1	2006-02-15 21:30:53	266
10	2005-05-25 00:02:21	1,824	399	2005-05-31 22:44:21	2	2006-02-15 21:30:53	266
11	2005-05-25 00:09:02	4,443	142	2005-06-02 20:56:02	2	2006-02-15 21:30:53	266

EXTRACT() Function

+ **Definition:** The **EXTRACT()** function extracts a part from a given date.

+ **Syntax:**

```
EXTRACT(part FROM date)
```



EXTRACT() Function

+ Parameter:

Parameter	Description
part	Required. The part to extract. Can be one of the following: • MICROSECOND • SECOND • MINUTE • HOUR • DAY • WEEK • MONTH • QUARTER • YEAR • SECOND_MICROSECOND • MINUTE_MICROSECOND • MINUTE_SECOND • HOUR_MICROSECOND • HOUR_SECOND • HOUR_MINUTE • DAY_MICROSECOND • DAY_SECOND • DAY_MINUTE • DAY_HOUR • YEAR_MONTH
date	Required. The date to extract a part from

EXTRACT() Example

```
SELECT EXTRACT(MICROSECOND FROM '2022-08-30 09:45:20.000001');
```

123 EXTRACT(MICROSECOND FROM '2022-08-30 09:45:20.000001')
1

```
SELECT EXTRACT(SECOND FROM '2022-08-30 09:45:20');
```

123 EXTRACT(SECOND FROM '2022-08-30 09:45:20')
20

```
SELECT EXTRACT(MINUTE FROM '2022-08-30 09:45:20');
```

123 EXTRACT(MINUTE FROM '2022-08-30 09:45:20')
45

EXTRACT() Example (Cont.)

```
SELECT EXTRACT(HOUR FROM '2022-08-30 09:45:20');
```

123 EXTRACT(HOUR FROM '2022-08-30 09:45:20')
9

```
SELECT EXTRACT(DAY FROM '2022-08-30');
```

123 EXTRACT(DAY FROM '2022-08-30')
30

```
SELECT EXTRACT(WEEK FROM '2022-08-30');
```

123 EXTRACT(WEEK FROM '2022-08-30')
35

EXTRACT() Example (Cont.)

```
SELECT EXTRACT(QUARTER FROM '2022-08-30');
```

123 EXTRACT(QUARTER FROM '2022-08-30')
3

```
SELECT EXTRACT(YEAR FROM '2022-08-30');
```

123 EXTRACT(YEAR FROM '2022-08-30')
2,022

EXTRACT() Example (Cont.)

```
SELECT EXTRACT(SECOND_MICROSECOND FROM '2022-08-30 09:45:20.000001');
```

123 EXTRACT(SECOND_MICROSECOND FROM '2022-08-30 09:45:20.000001')
20,000,001

```
SELECT EXTRACT(MINUTE_MICROSECOND FROM '2022-08-30 09:45:20.000001');
```

123 EXTRACT(MINUTE_MICROSECOND FROM '2022-08-30 09:45:20.000001')
4,520,000,001

```
SELECT EXTRACT(MINUTE_SECOND FROM '2022-08-30 09:45:20');
```

123 EXTRACT(MINUTE_SECOND FROM '2022-08-30 09:45:20')
4,520

EXTRACT() Example (Cont.)

```
SELECT EXTRACT(HOUR_MICROSECOND FROM '2022-08-30 09:45:20.000001');
```

123 EXTRACT(HOUR_MICROSECOND FROM '2022-08-30 09:45:20.000001')
94,520,000,001

```
SELECT EXTRACT(HOUR_SECOND FROM '2022-08-30 09:45:20');
```

123 EXTRACT(HOUR_SECOND FROM '2022-08-30 09:45:20')
94,520

```
SELECT EXTRACT(HOUR_MINUTE FROM '2022-08-30 09:45:20');
```

123 EXTRACT(HOUR_MINUTE FROM '2022-08-30 09:45:20')
945

EXTRACT() Example (Cont.)

```
SELECT EXTRACT(DAY_MICROSECOND FROM '2022-08-30 09:45:20.000001');
```

123 EXTRACT(DAY_MICROSECOND FROM '2022-08-30 09:45:20.000001')
30,094,520,000,001

```
SELECT EXTRACT(DAY_SECOND FROM '2022-08-30 09:45:20');
```

123 EXTRACT(DAY_SECOND FROM '2022-08-30 09:45:20')
30,094,520

```
SELECT EXTRACT(DAY_MINUTE FROM '2022-08-30 09:45:20');
```

123 EXTRACT(DAY_MINUTE FROM '2022-08-30 09:45:20')
300,945

EXTRACT() Example (Cont.)

```
SELECT EXTRACT(DAY_HOUR FROM '2022-08-30 09:45:20');
```

123 EXTRACT(DAY_HOUR FROM '2022-08-30 09:45:20')
3,009

```
SELECT EXTRACT(YEAR_MONTH FROM '2022-08-30');
```

123 EXTRACT(YEAR_MONTH FROM '2022-08-30')
202,208

DATE_FORMAT() Function

+ Definition: The **DATE_FORMAT()** function formats a date as specified.

+ Syntax:

```
DATE_FORMAT(date, format)
```

+ Parameter:

Parameter	Description
date	Required. The date to be formatted
format	Required. The format to use. Can be one or a combination of the following values:

DATE_FORMAT() Function (Cont.)

+ Parameter:

Format	Description
%a	Abbreviated weekday name (Sun to Sat)
%b	Abbreviated month name (Jan to Dec)
%c	Numeric month name (0 to 12)
%D	Day of the month as a numeric value, followed by suffix (1st, 2nd, 3rd, ...)
%d	Day of the month as a numeric value (01 to 31)
%e	Day of the month as a numeric value (0 to 31)
%f	Microseconds (000000 to 999999)
%H	Hour (00 to 23)
%h	Hour (00 to 12)
%I	Hour (00 to 12)
%i	Minutes (00 to 59)
%j	Day of the year (001 to 366)
%k	Hour (0 to 23)
%l	Hour (1 to 12)
%M	Month name in full (January to December)
%m	Month name as a numeric value (00 to 12)

DATE_FORMAT() Function (Cont.)

+ Parameter:

Format	Description
%p	AM or PM
%r	Time in 12 hour AM or PM format (hh:mm:ss AM/PM)
%S	Seconds (00 to 59)
%s	Seconds (00 to 59)
%T	Time in 24 hour format (hh:mm:ss)
%U	Week where Sunday is the first day of the week (00 to 53)
%u	Week where Monday is the first day of the week (00 to 53)
%V	Week where Sunday is the first day of the week (01 to 53). Used with %X
%v	Week where Monday is the first day of the week (01 to 53). Used with %x
%W	Weekday name in full (Sunday to Saturday)
%w	Day of the week where Sunday=0 and Saturday=6
%X	Year for the week where Sunday is the first day of the week. Used with %V
%x	Year for the week where Monday is the first day of the week. Used with %v
%Y	Year as a numeric, 4-digit value
%y	Year as a numeric, 2-digit value

DATE_FORMAT() Example

```
SELECT DATE_FORMAT('2022-08-30', '%Y');
```

DATE_FORMAT('2022-08-30', '%Y')
2022

```
SELECT DATE_FORMAT('2022-08-30', '%M %d, %Y');
```

DATE_FORMAT('2022-08-30', '%M %d, %Y')
August 30, 2022

```
SELECT DATE_FORMAT('2022-08-30', '%M %e %Y');
```

DATE_FORMAT('2022-08-30', '%M %e %Y')
August 30 2022

DATE_FORMAT() Example (Cont.)

```
SELECT DATE_FORMAT('2022-08-30', '%W, %M %e, %Y');
```

DATE_FORMAT('2022-08-30', '%W, %M %e, %Y')
Tuesday, August 30, 2022

```
SELECT DATE_FORMAT('2022-08-30', '%W');
```

DATE_FORMAT('2022-08-30', '%W')
Tuesday

STR_TO_DATE() Function

+ **Definition:** The **STR_TO_DATE()** function returns a date based on a string and a format..

+ **Syntax:**

```
STR_TO_DATE(string, format)
```

+ **Parameter:**

Parameter	Description
string	Required. The string to be formatted to a date
format	Required. The format to use. Can be one or a combination of the following values:

STR_TO_DATE() () Function (Cont.)

+ Parameter:

Format	Description
%a	Abbreviated weekday name (Sun to Sat)
%b	Abbreviated month name (Jan to Dec)
%c	Numeric month name (0 to 12)
%D	Day of the month as a numeric value, followed by suffix (1st, 2nd, 3rd, ...)
%d	Day of the month as a numeric value (01 to 31)
%e	Day of the month as a numeric value (0 to 31)
%f	Microseconds (000000 to 999999)
%H	Hour (00 to 23)
%h	Hour (00 to 12)
%I	Hour (00 to 12)
%i	Minutes (00 to 59)
%j	Day of the year (001 to 366)
%k	Hour (0 to 23)
%l	Hour (1 to 12)
%M	Month name in full (January to December)
%m	Month name as a numeric value (00 to 12)

STR_TO_DATE() Function (Cont.)

+ Parameter:

Format	Description
%p	AM or PM
%r	Time in 12 hour AM or PM format (hh:mm:ss AM/PM)
%S	Seconds (00 to 59)
%s	Seconds (00 to 59)
%T	Time in 24 hour format (hh:mm:ss)
%U	Week where Sunday is the first day of the week (00 to 53)
%u	Week where Monday is the first day of the week (00 to 53)
%V	Week where Sunday is the first day of the week (01 to 53). Used with %X
%v	Week where Monday is the first day of the week (01 to 53). Used with %x
%W	Weekday name in full (Sunday to Saturday)
%w	Day of the week where Sunday=0 and Saturday=6
%X	Year for the week where Sunday is the first day of the week. Used with %V
%x	Year for the week where Monday is the first day of the week. Used with %v
%Y	Year as a numeric, 4-digit value
%y	Year as a numeric, 2-digit value

STR_TO_DATE() () Example

```
SELECT STR_TO_DATE('August 30 2022', '%M %d %Y');
```

STR_TO_DATE('August 30 2022', '%M %d %Y')
2022-08-30

```
SELECT STR_TO_DATE('August, 30,2022', '%M,%e,%Y');
```

STR_TO_DATE('August, 30,2022', '%M,%e,%Y')
2022-08-30

```
SELECT STR_TO_DATE('Tuesday, August 30,2022', '%W, %M %e, %Y');
```

STR_TO_DATE('Tuesday, August 30,2022', '%W, %M %e, %Y')
2022-08-30

STR_TO_DATE() () Example (Cont.)

```
SELECT STR_TO_DATE('2022,9,30 08', '%Y,%m,%d %h');
```

STR_TO_DATE('2022,9,30 08', '%Y,%m,%d %h')
2022-09-30 08:00:00

```
SELECT STR_TO_DATE('2022,8,30 02,30,05', '%Y,%m,%d %h,%i,%s');
```

STR_TO_DATE('2022,8,30 02,30,05', '%Y,%m,%d %h,%i,%s')
2022-08-30 02:30:05

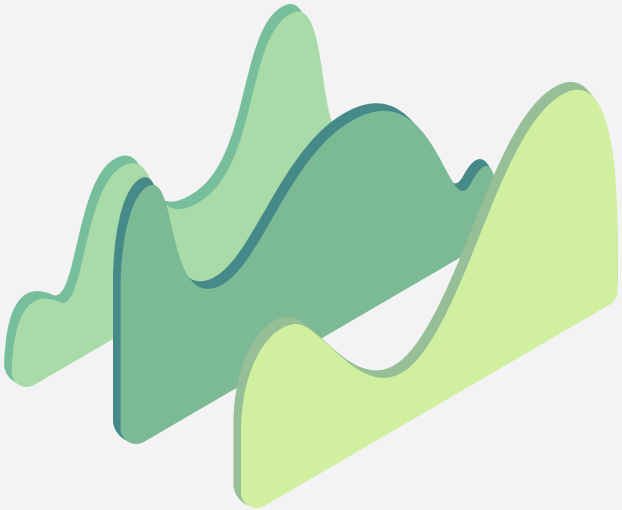
```
SELECT STR_TO_DATE('11,10,35', '%h,%i,%s');
```

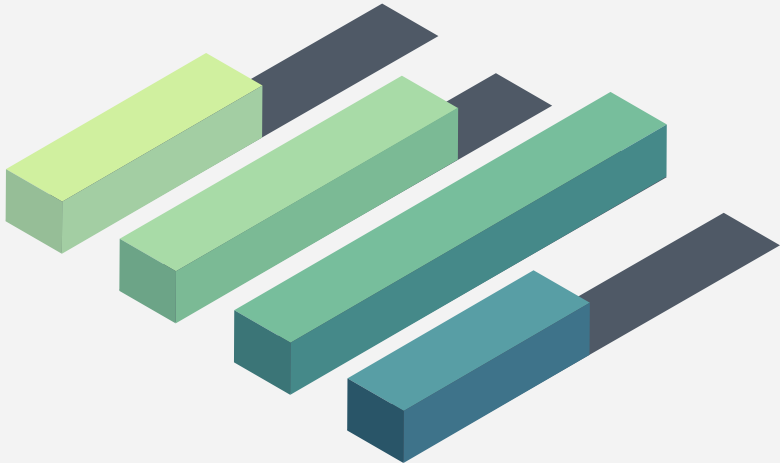
STR_TO_DATE('11,10,35', '%h,%i,%s')
11:10:35

200Lab Education

Arithmetic Operators

Van Anh - Data Mentor



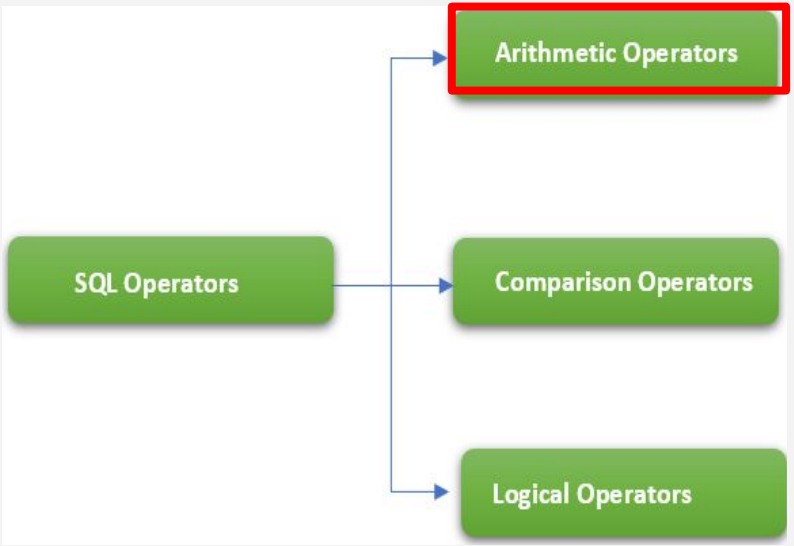


Definitions

+ In SQL, the arithmetic operators are used to perform mathematical operations on the numerical values

+ We can use these operators with the SELECT, WHERE statements in SQL.

+ These types of operators are used between two or more numerical operands for performing addition, subtraction, multiplication, and division operations.



Arithmetic Operators In SQL

SQL Arithmetic Operators

Operator	Description
+	Add
-	Subtract
*	Multiply
/	Divide
%	Modulo

Arithmetic Operators In SQL

+ Syntax:

```
SELECT <Expression>[arithmetic operator]<expression>...
 FROM [table_name]
 WHERE [expression];
```

+ Parameter:

Parameter	Description
Expression	Expression made up of a single constant, variable, scalar function, or column name and can also be the pieces of a SQL query that compare values against other values or perform arithmetic calculations.
arithmetic operator	Plus(+), minus(-), multiply(*), and divide(/).
table_name	Name of the table.

Arithmetic Operators In SQL Example

+ Add operator:

```
SELECT 20 + 30;
```

20 + 30
50

```
SELECT
    store_id,
    manager_staff_id,
    store_id + manager_staff_id AS test_add
FROM sakila.store;
```

store_id	manager_staff_id	test_add
1	1	2
2	2	4

Arithmetic Operators In SQL Example

+ Subtract operator:

```
SELECT 3.56 - 4.32;
```

123 3.56 - 4.32
-0.76

```
SELECT
    store_id,
    manager_staff_id,
    store_id - manager_staff_id AS test_sub
FROM sakila.store;
```

store_id	manager_staff_id	test_sub
1	1	0
2	2	0

Arithmetic Operators In SQL Example

+ Multiply operator:

```
SELECT 2.33 * 5;
```

123 2.33 * 5
11.65

```
SELECT
store_id,
manager_staff_id,
store_id * manager_staff_id AS test_multiply
FROM sakila.store;
```

store_id	manager_staff_id	test_multiply
1	1	1
2	2	4

Arithmetic Operators In SQL Example

+ Divide operator:

```
SELECT 900/3;
```

900/3
300

```
SELECT
    store_id,
    manager_staff_id,
    store_id/manager_staff_id AS test_divide
FROM sakila.store;
```

store_id	manager_staff_id	test_divide
1	1	1
2	2	1

Arithmetic Operators In SQL Example

+ Modulo operator:

```
SELECT 901 % 3;
```

901%3
1

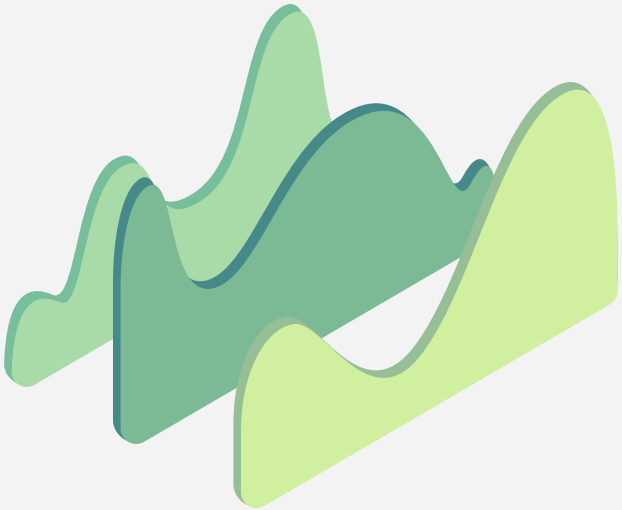
```
SELECT
    store_id,
    manager_staff_id,
    store_id % manager_staff_id AS test_modulo
FROM sakila.store;
```

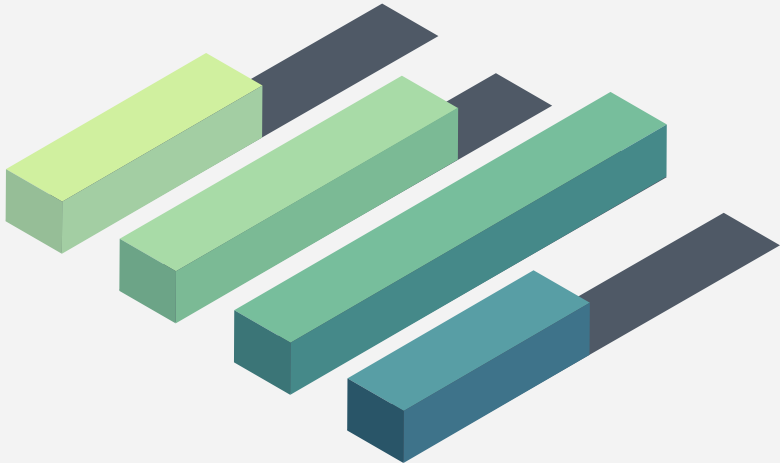
store_id	manager_staff_id	test_modulo
1	1	0
2	2	0

200Lab Education

Aggregation Functions

Van Anh - Data Mentor



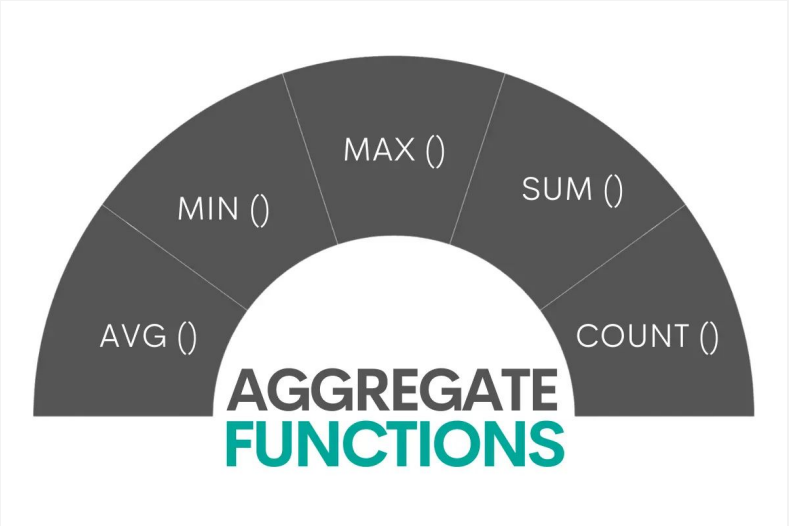


Definitions

+ An aggregate function is a function that performs calculations on a group of values and returns a single value.

+ It is used to summarize the data. It performs calculations on multiple rows of a single column of a table to form a single value of more significant meaning.

+ Aggregate functions are often used with the GROUP BY and HAVING clauses of the SELECT statement where null values are ignored (except for COUNT(*))



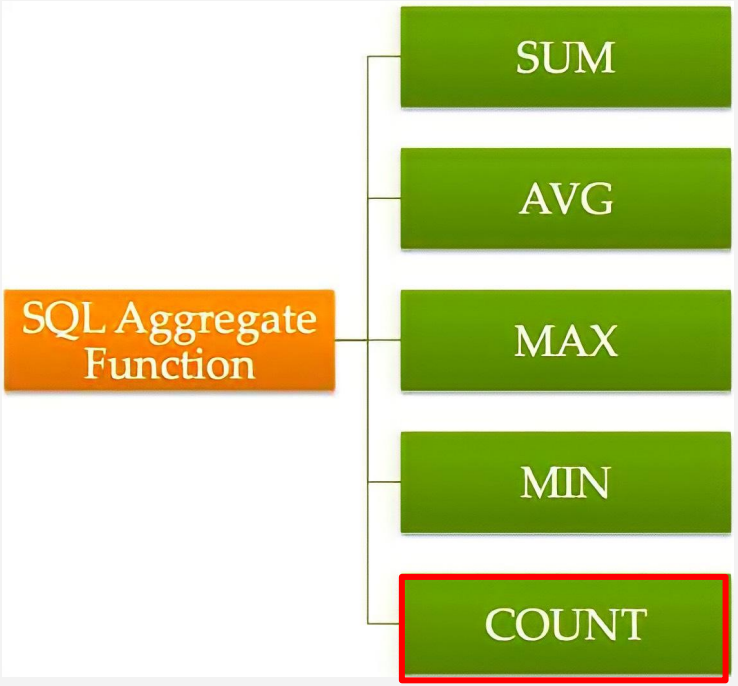
Count Function

+ Definitions: The **COUNT()** function returns the number of rows that matches a specified criterion.

+ Syntax:

```
SELECT COUNT(column_name)
FROM table_name
WHERE condition;
```

```
SELECT COUNT (*)
FROM table_name
WHERE condition;
```



+ Note: ***NULL*** *values are not counted in count a column and counted in count(*)*

Count Function Examples

```sql
SELECT
    COUNT(*)
FROM sakila.address;
```

123 COUNT(*)
603

```sql
SELECT
    COUNT(address2)
FROM sakila.address;
```

123 COUNT(address2)
599

```sql
SELECT
    COUNT(address2) AS count_not_null
FROM sakila.address
WHERE address2 IS NOT NULL;
```

123 count_not_null
599

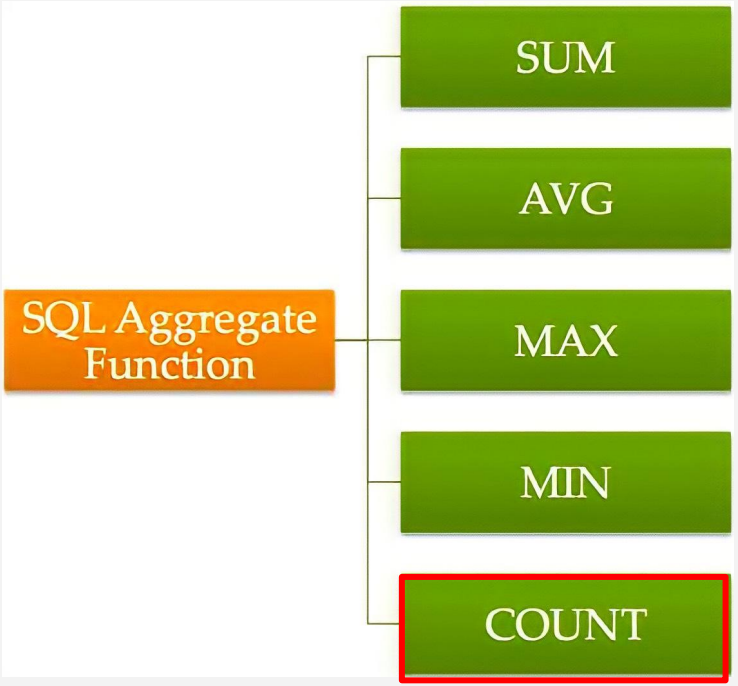
Count Distinct Function

+ Definitions: The **COUNT(DISTINCT)** function returns the number of unique rows that matches a specified criterion.

+ Syntax:

```
SELECT COUNT (DISTINCT column_name)
FROM table_name
WHERE condition;
```

+ Note: ***NULL*** *values are not counted.*



Count Distinct Function Examples

```
SELECT
    COUNT(actor_id) actors,
    COUNT(film_id) AS films
FROM sakila.film_actor;
```

123 actors	123 films
5,462	5,462

```
SELECT
    COUNT( DISTINCT actor_id) total_actors,
    COUNT(DISTINCT film_id) AS total_films
FROM sakila.film_actor;
```

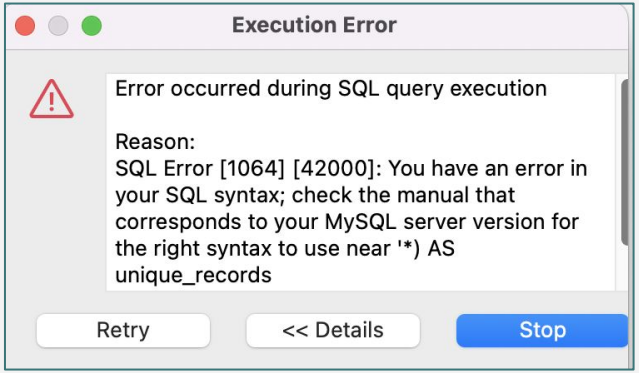
123 total_actors	123 total_films
200	997

Count Distinct Function Examples

```
SELECT
    COUNT( DISTINCT actor_id) total_actors,
    COUNT(DISTINCT film_id) AS total_films
FROM sakila.film_actor
WHERE film_id >= 100;
```

123 total_actors	123 total_films
200	898

```
SELECT
    COUNT(DISTINCT *) AS unique_records
FROM sakila.film_actor;
```

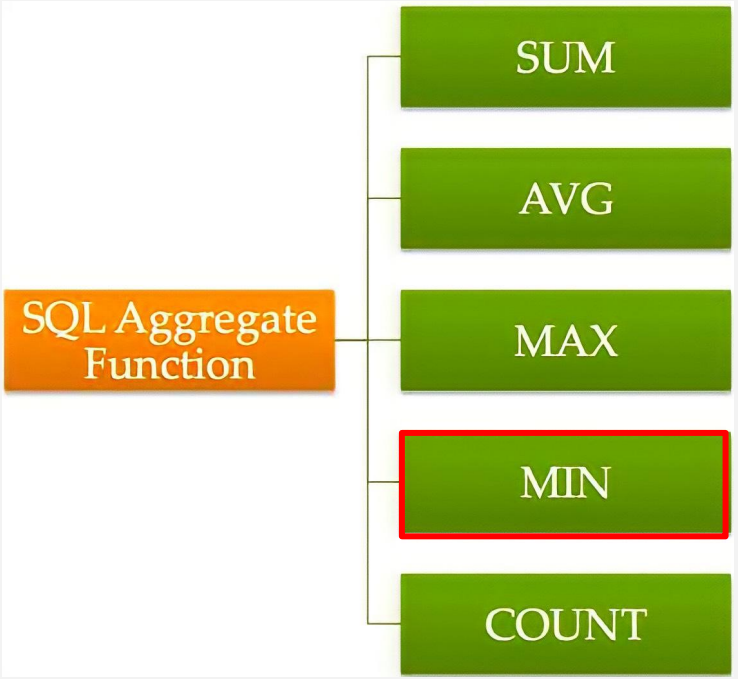


Minimum Function

+ **Definitions:** The **MIN()** function returns the smallest value of the selected column.

+ **Syntax:**

```
SELECT MIN(column_name)
FROM table_name
WHERE condition;
```



+ **Note:** ***NULL*** *values are ignored.*

Minimum Function Examples

```
SELECT
    MIN(amount) AS 'Minimum Amount'
FROM sakila.payment;
```

123 Minimum Amount
0

```
SELECT
    MIN(amount) AS 'Minimum Amount'
FROM sakila.payment
WHERE payment_date >= '2005-08-01 00:00:00'
AND payment_date <= '2005-12-31 23:59:59';
```

123 Minimum Amount
0.99

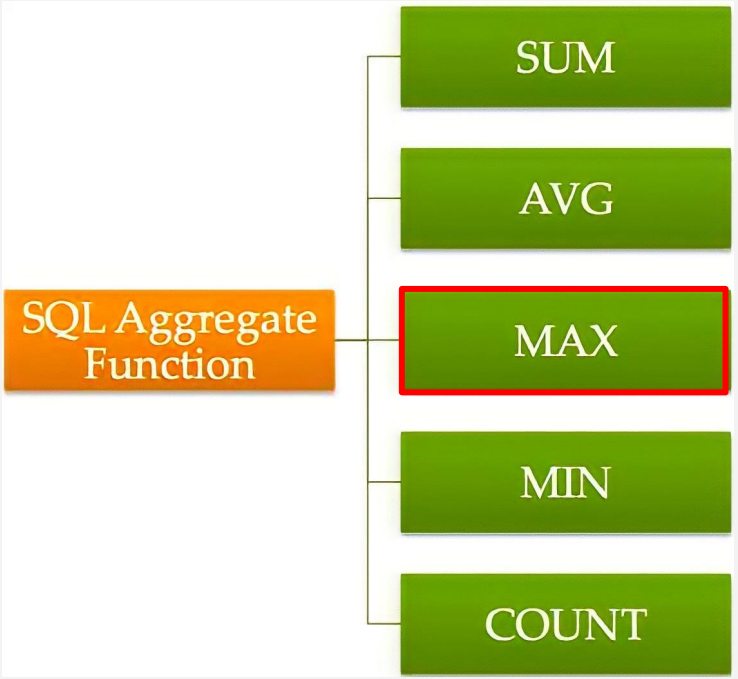
Maximum Function

+ Definitions: The **MAX()** function returns the largest value of the selected column.

+ Syntax:

```
SELECT MAX(column_name)
FROM table_name
WHERE condition;
```

+ Note: ***NULL*** *values are ignored.*



Maximum Function Examples

```sql
SELECT
    MAX(amount) AS 'Maximum Amount'
FROM sakila.payment;
```

123 Maximum Amount
11.99

```sql
SELECT
    MAX(amount) AS 'Maximum Amount'
FROM sakila.payment
WHERE customer_id = 1;
```

123 Maximum Amount
9.99

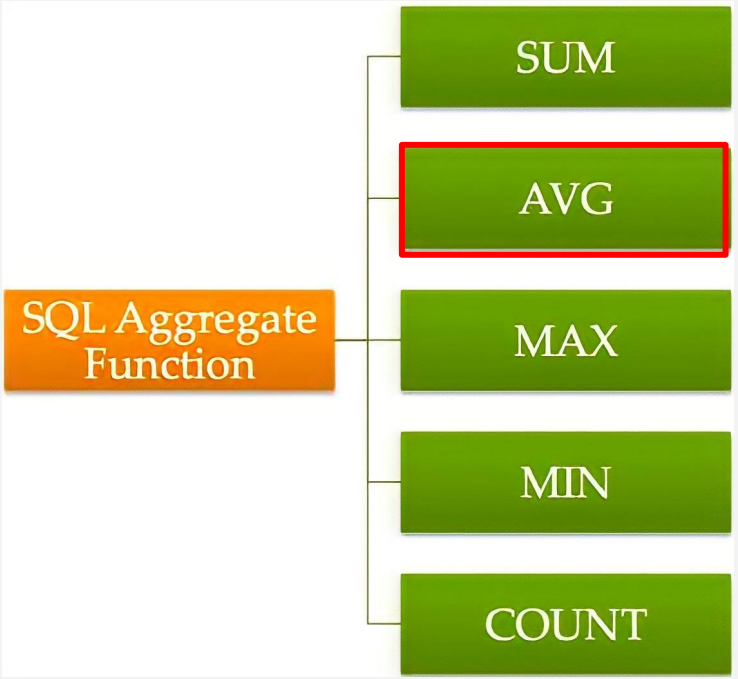
Average Function

+ Definitions: The **AVG()** function returns the average value of a numeric column.

+ Syntax:

```
SELECT AVG(column_name)
FROM table_name
WHERE condition;
```

+ Note: ***NULL*** *values are ignored.*



Average Function Examples

```
SELECT
    AVG(replacement_cost) AS `Average Replacement Cost`
FROM sakila.film;
```

123 Average Replacement Cost
19.984

```
SELECT
    AVG(replacement_cost) AS `Average Replacement Cost`
FROM sakila.film
WHERE `length` >= 100;
```

123 Average Replacement Cost
20.007685

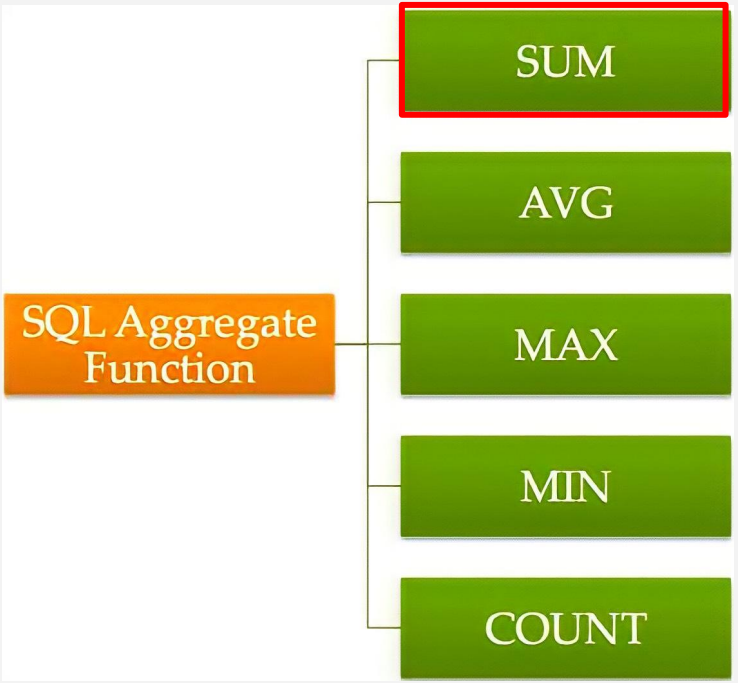
Sum Function

+ Definitions: The **SUM()** function returns the total sum of a numeric column.

+ Syntax:

```
SELECT SUM(column_name)
FROM table_name
WHERE condition;
```

+ Note: ***NULL*** *values are ignored.*



Sum Function Examples

```sql
SELECT
    SUM(replacement_cost) AS `Total Replacement Cost`
FROM sakila.film;
```

123 Total Replacement Cost
19,984

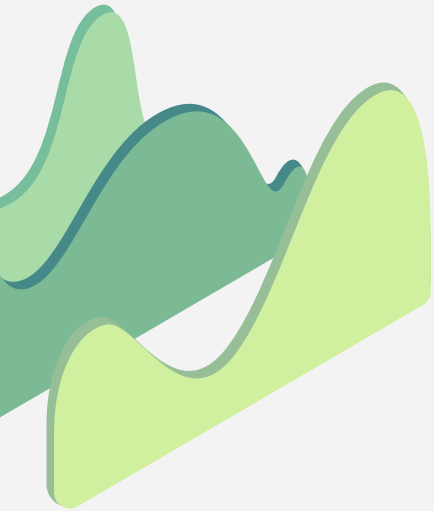
```sql
SELECT
    SUM(replacement_cost) AS `Total Replacement Cost`
FROM sakila.film
WHERE `length` <= 100;
```

123 Total Replacement Cost
7,791.1

200Lab Education

Understanding Grouping in SQL

Van Anh - Data Mentor







What is Group By

Definitions & Syntax

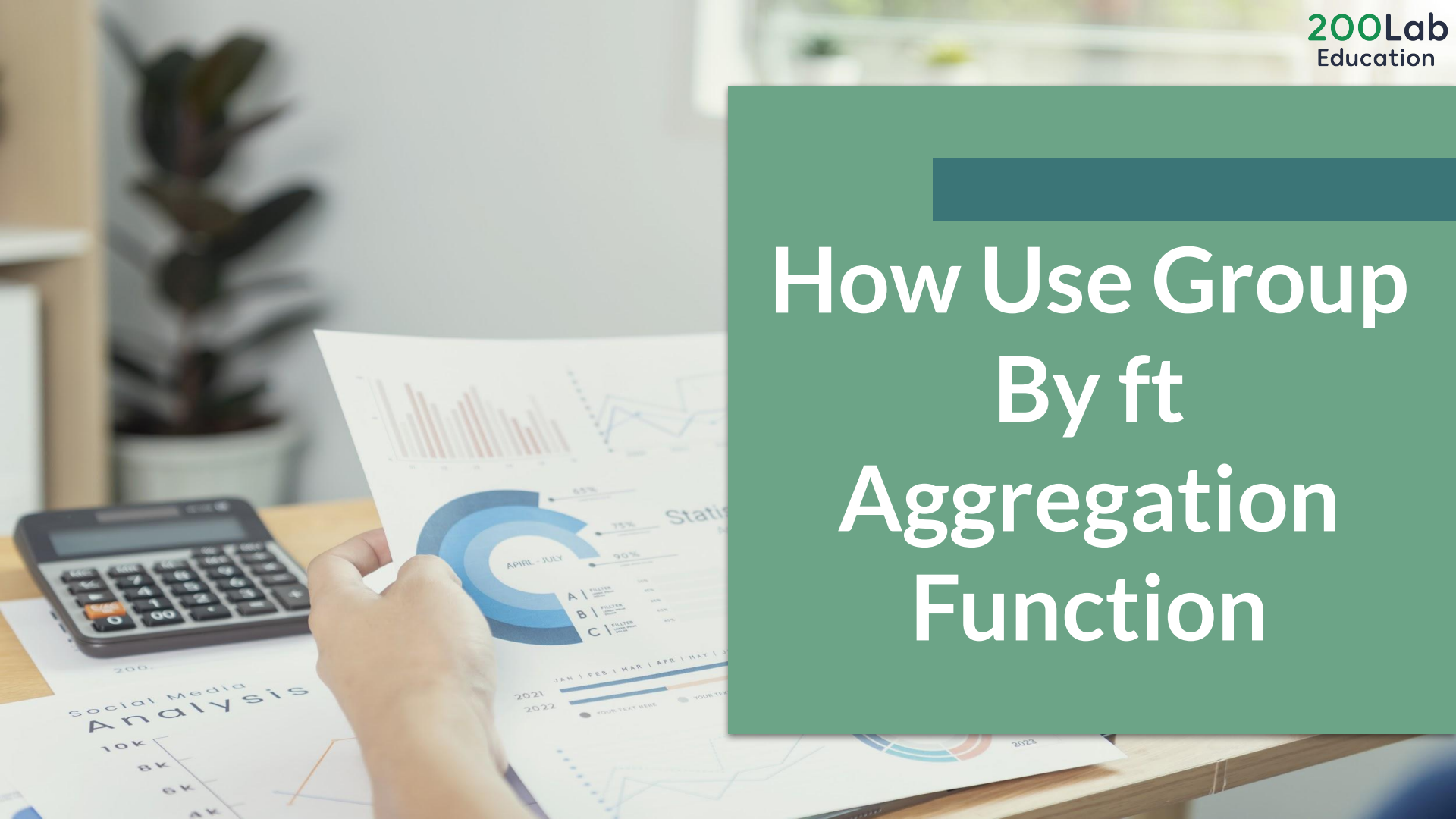
+ In SQL, a **GROUP BY** clause is used to group the rows together. So when you use aggregate function on a column, the result describes the data for that specific group of rows.

+ GROUP BY is used for data aggregation, using aggregate functions. such as ***SUM()***, ***MIN()***, ***MAX()***, ***AVG()*** and ***COUNT()*** .

+ Syntax:

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
WHERE column_name operator value
GROUP BY column_name;
```



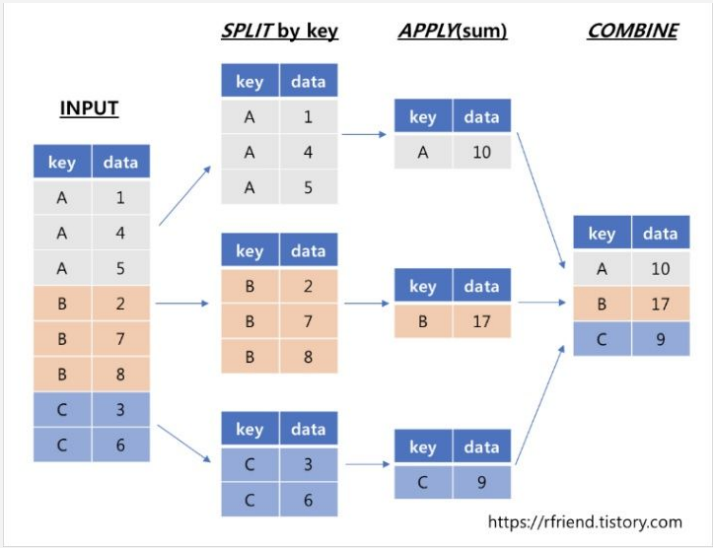


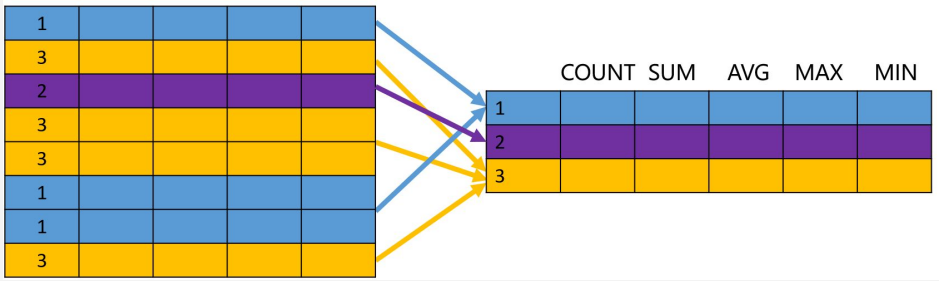
How Use Group By ft Aggregation Function

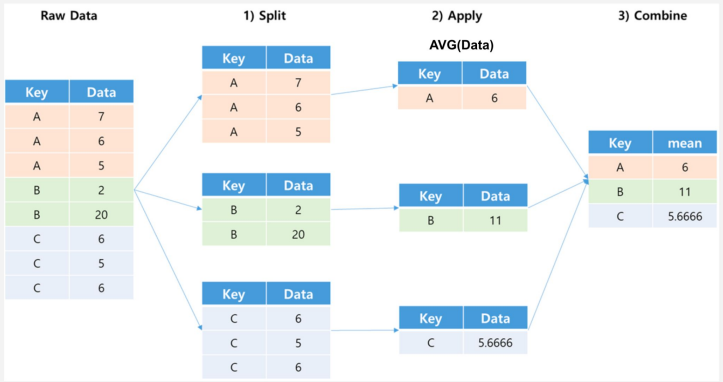
Control Flow

The SQL Group By statement uses the split-apply-combine strategy:

- **Split:** The different groups are split with their values.
- **Apply:** The aggregate function is applied to the values of these groups.
- **Combine:** The values are combined in a single row.







Group By Example

1. Use **GROUP BY** on single column:

```
SELECT
    last_name,
    COUNT(*)
FROM sakila.actor
GROUP BY last_name;
```

	last_name	COUNT(*)
1	AKROYD	3
2	ALLEN	3
3	ASTAIRE	1
4	BACALL	1
5	BAILEY	2
6	BALE	1
7	BALL	1
8	BARRYMORE	1
9	BASINGER	1
10	BENING	2
11	BERGEN	1
12	BERGMAN	1
13	BERRY	3
14	BIRCH	1
15	BLOOM	1
16	BOLGER	2

```
SELECT
    customer_id,
    SUM(amount) AS `Total Amount`,
    MIN(amount) AS `Minimum Amount`,
    MAX(amount) AS `Maximum Amount`,
    AVG(amount) AS `Average Amount`,
    COUNT(DISTINCT payment_id) AS `Payment Frequency`
FROM sakila.payment
GROUP BY 1;
```

	customer_id	Total Amount	Minimum Amount	Maximum Amount	Average Amount	Payment Frequency
1	1	118.68	0.99	9.99	3.70875	32
2	2	128.73	0.99	10.99	4.767778	27
3	3	135.74	0.99	10.99	5.220769	26
4	4	81.78	0.99	8.99	3.717273	22
5	5	144.62	0.99	9.99	3.805789	38
6	6	93.72	0.99	7.99	3.347143	28
7	7	151.67	0.99	8.99	4.596061	33
8	8	92.76	0.99	9.99	3.865	24
9	9	89.77	0.99	7.99	3.903043	23
10	10	99.75	0.99	8.99	3.99	25
11	11	106.76	0.99	9.99	4.448333	24

Group By Example

2. Use **GROUP BY** on multiple columns:

	payment_id	customer_id	staff_id	rental_id	amount	payment_date	last_update
2	2	1	1	573	0.99	2005-05-28 10:35:23	2006-02-15 22:12:30
3	3	1	1	1,185	5.99	2005-06-15 00:54:12	2006-02-15 22:12:30
4	4	1	2	1,422	0.99	2005-06-15 18:02:53	2006-02-15 22:12:30
5	5	1	2	1,476	9.99	2005-06-15 21:08:46	2006-02-15 22:12:30
6	6	1	1	1,725	4.99	2005-06-16 15:18:57	2006-02-15 22:12:30
7	7	1	1	2,308	4.99	2005-06-18 08:41:48	2006-02-15 22:12:30
8	8	1	2	2,363	0.99	2005-06-18 13:33:59	2006-02-15 22:12:30
9	9	1	1	3,284	3.99	2005-06-21 06:24:45	2006-02-15 22:12:30
10	10	1	2	4,526	5.99	2005-07-08 03:17:05	2006-02-15 22:12:30
11	11	1	1	4,611	5.99	2005-07-08 07:33:56	2006-02-15 22:12:30
12	12	1	1	5,244	4.99	2005-07-09 13:24:07	2006-02-15 22:12:30
13	13	1	1	5,326	4.99	2005-07-09 16:38:01	2006-02-15 22:12:30
14	14	1	1	6,163	7.99	2005-07-11 10:13:46	2006-02-15 22:12:30
15	15	1	2	7,273	2.99	2005-07-27 11:31:22	2006-02-15 22:12:30
16	16	1	1	7,841	4.99	2005-07-28 09:04:45	2006-02-15 22:12:30
17	17	1	2	8,033	4.99	2005-07-28 16:18:23	2006-02-15 22:12:30
18	18	1	1	8,074	0.99	2005-07-28 17:33:39	2006-02-15 22:12:30
19	19	1	2	8,116	0.99	2005-07-28 19:20:07	2006-02-15 22:12:30
20	20	1	2	8,326	2.99	2005-07-29 03:58:49	2006-02-15 22:12:30
21	21	1	2	9,571	2.99	2005-07-31 02:42:18	2006-02-15 22:12:30
22	22	1	2	10,437	4.99	2005-08-01 08:51:04	2006-02-15 22:12:30
23	23	1	2	11,299	3.99	2005-08-02 15:36:52	2006-02-15 22:12:30
24	24	1	1	11,367	0.99	2005-08-02 18:01:38	2006-02-15 22:12:30
25	25	1	2	11,824	4.99	2005-08-17 12:37:54	2006-02-15 22:12:30
26	26	1	1	12,250	0.99	2005-08-18 03:57:29	2006-02-15 22:12:30
27	27	1	2	13,068	0.99	2005-08-19 09:55:16	2006-02-15 22:12:30
28	28	1	2	13,176	2.99	2005-08-19 13:56:54	2006-02-15 22:12:30
29	29	1	1	14,762	0.99	2005-08-21 23:33:57	2006-02-15 22:12:30
30	30	1	1	14,825	1.99	2005-08-22 01:27:57	2006-02-15 22:12:30
31	31	1	2	15,298	2.99	2005-08-22 19:41:37	2006-02-15 22:12:30
32	32	1	1	15,315	5.99	2005-08-22 20:03:46	2006-02-15 22:12:30
33	33	2	1	320	4.99	2005-05-27 00:09:24	2006-02-15 22:12:30
34	34	2	1	2,128	2.99	2005-06-17 20:54:58	2006-02-15 22:12:30
35	35	2	1	5,636	2.99	2005-07-10 06:31:24	2006-02-15 22:12:30
36	36	2	1	5,755	6.99	2005-07-10 12:38:56	2006-02-15 22:12:30
37	37	2	2	7,346	4.99	2005-07-27 14:30:42	2006-02-15 22:12:30
38	38	2	1	7,376	5.99	2005-07-27 15:23:02	2006-02-15 22:12:30
39	39	2	2	7,459	5.99	2005-07-27 18:40:20	2006-02-15 22:12:30

Group By Example

2. Use **GROUP BY** on multiple columns:

```
SELECT
    customer_id,
    staff_id,
    SUM(amount) AS `Total Amount`,
    MIN(amount) AS `Minimum Amount`,
    MAX(amount) AS `Maximum Amount`,
    AVG(amount) AS `Average Amount`,
    COUNT(DISTINCT payment_id) AS `Payment Frequency`
FROM sakila.payment
GROUP BY 1,2;
```

	customer_id	staff_id	Total Amount	Minimum Amount	Maximum Amount	Average Amount	Payment Frequency
1	1	1	64.83	0.99	7.99	3.813529	17
2	1	2	53.85	0.99	9.99	3.59	15
3	2	1	60.85	0.99	6.99	4.056667	15
4	2	2	67.88	0.99	10.99	5.656667	12
5	3	1	64.86	1.99	8.99	4.632857	14
6	3	2	70.88	0.99	10.99	5.906667	12
7	4	1	49.88	0.99	6.99	4.156667	12
8	4	2	31.9	0.99	8.99	3.19	10
9	5	1	73.83	0.99	9.99	4.342941	17
10	5	2	70.79	0.99	6.99	3.370952	21
11	6	1	56.84	0.99	7.99	3.5525	16

Group By Example

3. Use **GROUP BY** with **ORDER BY**:

```sql
SELECT
    customer_id,
    staff_id,
    SUM(amount) AS `Total Amount`,
    MIN(amount) AS `Minimum Amount`,
    MAX(amount) AS `Maximum Amount`,
    AVG(amount) AS `Average Amount`,
    COUNT(DISTINCT payment_id) AS `Payment Frequency`
FROM sakila.payment
GROUP BY 1,2
ORDER BY 3 DESC;
```

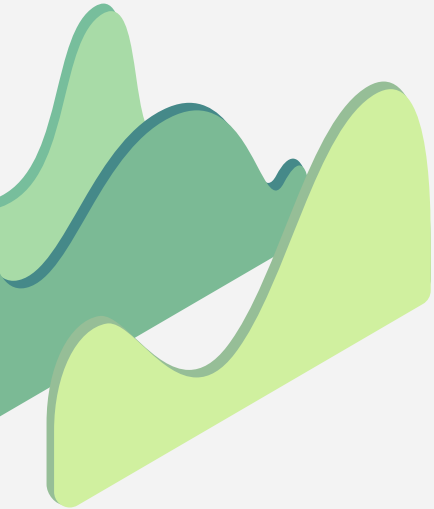
```sql
SELECT
    customer_id,
    staff_id,
    SUM(amount) AS `Total Amount`,
    MIN(amount) AS `Minimum Amount`,
    MAX(amount) AS `Maximum Amount`,
    AVG(amount) AS `Average Amount`,
    COUNT(DISTINCT payment_id) AS `Payment Frequency`
FROM sakila.payment
GROUP BY 1,2
ORDER BY SUM(amount) DESC;
```

	customer_id	staff_id	Total Amount	Minimum Amount	Maximum Amount	Average Amount	Payment Frequency
2	137	1	115.77	0.99	9.99	5.033478	23
3	526	1	111.79	0.99	10.99	5.323333	21
4	187	2	110.81	0.99	10.99	5.832105	19
5	148	2	110.78	0.99	10.99	5.035455	22
6	526	2	109.76	0.99	8.99	4.573333	24
7	178	1	108.81	2.99	10.99	5.726842	19
8	522	2	108.79	0.99	9.99	5.180476	21
9	459	1	108.78	0.99	9.99	4.944545	22
10	211	2	108.77	0.99	8.99	4.72913	23
11	148	1	105.76	0.99	10.99	4.406667	24
12	309	2	104.81	0.99	9.99	5.516316	19
13	26	1	104.78	0.99	9.99	4.762727	22
14	144	2	102.8	0.99	9.99	5.14	20

200Lab Education

Grouping Data With Conditions

Van Anh - Data Mentor



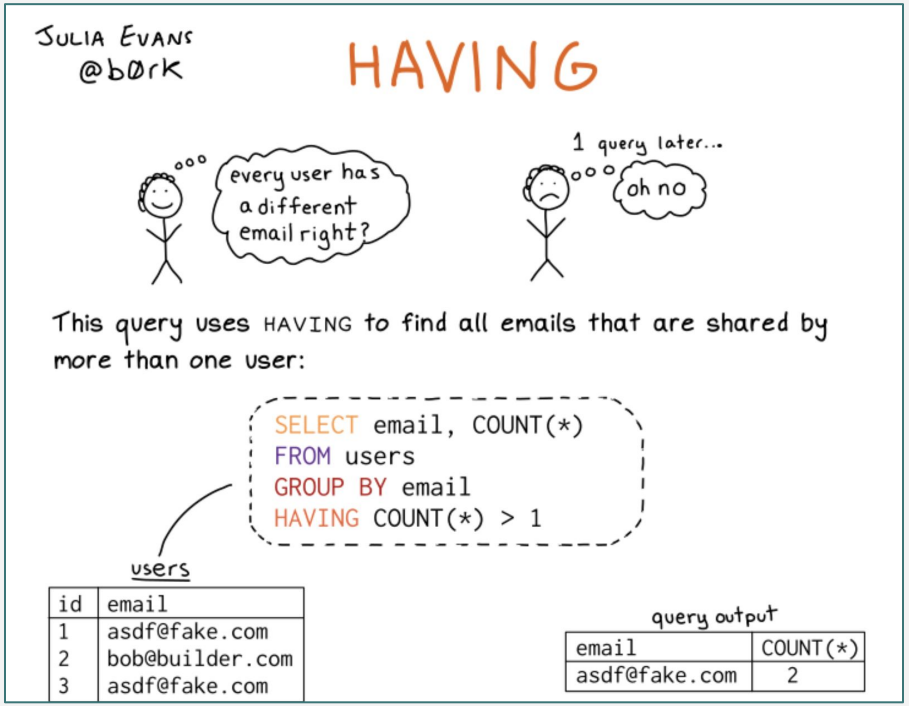


Definitions & Syntax of HAVING

- **HAVING** is like **WHERE** but operates on grouped records.
- Requires that a **GROUP BY** clause is present.
- Groups that meet the **HAVING** criteria will be returned.
- Used with aggregation functions : ***COUNT, MAX, MIN, SUM, AVG***

+ Syntax:

```
SELECT expression1, expression2, ... expression_n,
       aggregate_function (aggregate_expression)
FROM tables
[WHERE conditions]
GROUP BY expression1, expression2, ... expression_n
HAVING condition;
```



HAVING with WHERE

HAVING	WHERE
1. The HAVING clause is used in database systems to fetch the data/values from the groups according to the given condition.	1. The WHERE clause is used in database systems to fetch the data/values from the tables according to the given condition.
2. The HAVING clause is always executed with the GROUP BY clause.	2. The WHERE clause can be executed without the GROUP BY clause.
3. The HAVING clause can include SQL aggregate functions in a query or statement.	3. We cannot use the SQL aggregate function with WHERE clause in statements.
4. We can only use SELECT statement with HAVING clause for filtering the records.	4. Whereas, we can easily use WHERE clause with UPDATE, DELETE, and SELECT statements.
5. The HAVING clause is used in SQL queries after the GROUP BY clause.	5. The WHERE clause is always used before the GROUP BY clause in SQL queries.
6. We can implements this SQL clause in column operations.	6. We can implements this SQL clause in row operations.
7. It is a post-filter.	7. It is a pre-filter.
8. It is used to filter groups.	8. It is used to filter the single record of the table.



HAVING Example

```sql
SELECT
    customer_id,
    staff_id,
    SUM(amount) AS `Total Amount`,
    MIN(amount) AS `Minimum Amount`,
    MAX(amount) AS `Maximum Amount`,
    AVG(amount) AS `Average Amount`,
    COUNT(DISTINCT payment_id) AS `Payment Frequency`
FROM sakila.payment
GROUP BY 1,2
HAVING `Total Amount` >= 100;
```

customer_id	staff_id	Total Amount	Minimum Amount	Maximum Amount	Average Amount	Payment Frequency
26	1	104.78	0.99	9.99	4.762727	22
137	1	115.77	0.99	9.99	5.033478	23
144	2	102.8	0.99	9.99	5.14	20
148	1	105.76	0.99	10.99	4.406667	24
148	2	110.78	0.99	10.99	5.035455	22
176	1	126.74	0.99	8.99	4.874615	26
178	1	108.81	2.99	10.99	5.726842	19
187	2	110.81	0.99	10.99	5.832105	19
211	2	108.77	0.99	8.99	4.72913	23
236	2	100.78	0.99	8.99	4.580909	22
237	2	100.79	0.99	11.99	4.799524	21
309	2	104.81	0.99	9.99	5.516316	19
410	1	101.76	0.99	10.99	4.24	24
459	1	108.78	0.99	9.99	4.944545	22

```sql
SELECT
    customer_id,
    staff_id,
    SUM(amount) AS `Total Amount`,
    MIN(amount) AS `Minimum Amount`,
    MAX(amount) AS `Maximum Amount`,
    AVG(amount) AS `Average Amount`,
    COUNT(DISTINCT payment_id) AS `Payment Frequency`
FROM sakila.payment
GROUP BY 1,2
HAVING staff_id = 1;
```

customer_id	staff_id	Total Amount	Minimum Amount	Maximum Amount	Average Amount	Payment Frequency
1	1	64.83	0.99	7.99	3.813529	17
2	1	60.85	0.99	6.99	4.056667	15
3	1	64.86	1.99	8.99	4.632857	14
4	1	49.88	0.99	6.99	4.156667	12
5	1	73.83	0.99	9.99	4.342941	17
6	1	56.84	0.99	7.99	3.5525	16
7	1	80.82	0.99	8.99	4.49	18
8	1	57.86	0.99	9.99	4.132857	14
9	1	39.88	0.99	5.99	3.323333	12
10	1	40.88	0.99	7.99	3.406667	12
11	1	60.87	0.99	9.99	4.682308	13
12	1	31.9	0.99	5.99	3.19	10

HAVING Example (Cont.)

```sql
SELECT
    customer_id,
    staff_id,
    SUM(amount) AS `Total Amount`,
    MIN(amount) AS `Minimum Amount`,
    MAX(amount) AS `Maximum Amount`,
    AVG(amount) AS `Average Amount`,
    COUNT(DISTINCT payment_id) AS `Payment Frequency`
FROM sakila.payment
GROUP BY 1,2
HAVING `Total Amount` >= 100
AND customer_id >= 300;
```

customer_id	staff_id	Total Amount	Minimum Amount	Maximum Amount	Average Amount	Payment Frequency
309	2	104.81	0.99	9.99	5.516316	19
410	1	101.76	0.99	10.99	4.24	24
459	1	108.78	0.99	9.99	4.944545	22
468	2	100.8	0.99	10.99	5.04	20
469	1	100.79	0.99	10.99	4.799524	21
522	2	108.79	0.99	9.99	5.180476	21
526	1	111.79	0.99	10.99	5.323333	21
526	2	109.76	0.99	8.99	4.573333	24

HAVING Example (Cont.)

```sql
SELECT
    customer_id,
    staff_id,
    SUM(amount) AS `Total Amount`,
    MIN(amount) AS `Minimum Amount`,
    MAX(amount) AS `Maximum Amount`,
    AVG(amount) AS `Average Amount`,
    COUNT(DISTINCT payment_id) AS `Payment Frequency`
FROM sakila.payment
GROUP BY 1,2
HAVING `Total Amount` >= 100
OR `Maximum Amount` >= 10;
```

customer_id	staff_id	Total Amount	Minimum Amount	Maximum Amount	Average Amount	Payment Frequency
2	2	67.88	0.99	10.99	5.656667	12
3	2	70.88	0.99	10.99	5.906667	12
12	2	71.82	0.99	10.99	3.99	18
13	2	54.9	0.99	11.99	5.49	10
21	1	85.82	0.99	10.99	4.767778	18
26	1	104.78	0.99	9.99	4.762727	22
29	2	64.84	0.99	10.99	4.0525	16
33	1	70.84	0.99	10.99	4.4275	16
45	1	84.84	0.99	10.99	5.3025	16
49	2	75.85	0.99	10.99	5.056667	15
50	2	90.81	0.99	10.99	4.779474	19
54	1	67.85	0.99	10.99	4.523333	15

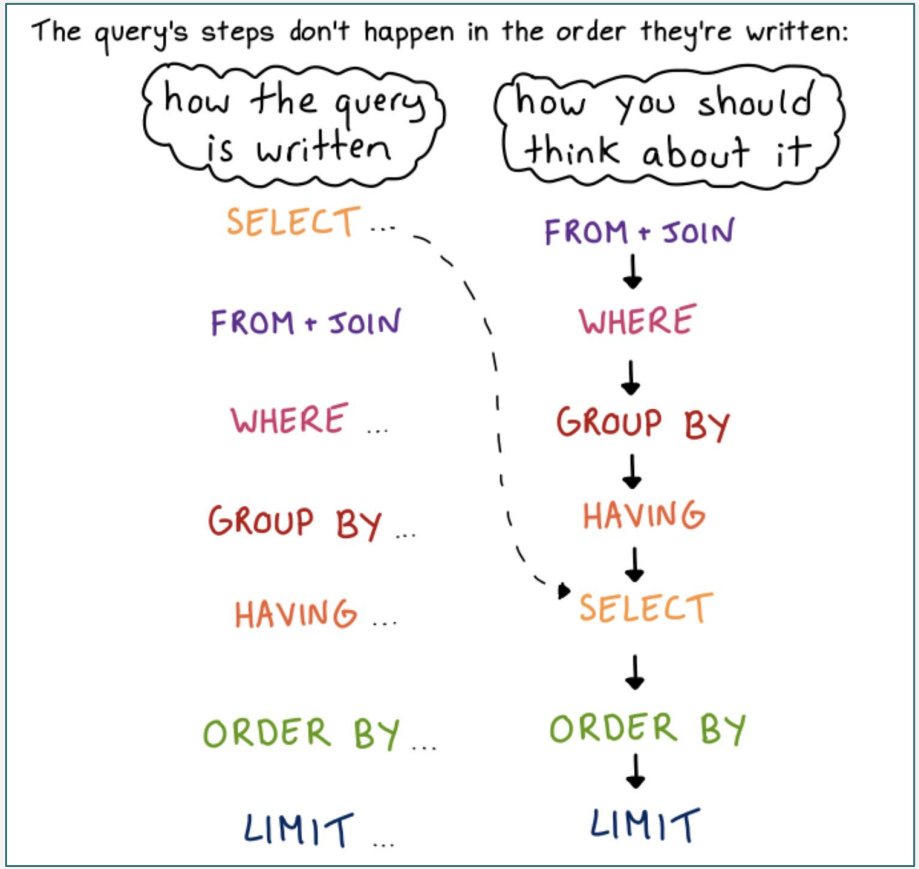
HAVING Example (Cont.)

```sql
SELECT
    customer_id,
    staff_id,
    SUM(amount) AS `Total Amount`,
    MIN(amount) AS `Minimum Amount`,
    MAX(amount) AS `Maximum Amount`,
    AVG(amount) AS `Average Amount`,
    COUNT(DISTINCT payment_id) AS `Payment Frequency`
FROM sakila.payment
GROUP BY 1,2
HAVING NOT `Total Amount` > 100
AND `Maximum Amount` >= 10
ORDER BY `Total Amount` DESC, `Payment Frequency` ASC;
```

customer_id	staff_id	Total Amount	Minimum Amount	Maximum Amount	Average Amount	Payment Frequency
307	1	96.77	0.99	10.99	4.207391	23
380	1	95.77	0.99	10.99	4.163913	23
300	1	94.81	0.99	10.99	4.99	19
50	2	90.81	0.99	10.99	4.779474	19
196	2	90.78	0.99	11.99	4.126364	22
470	1	89.82	0.99	10.99	4.99	18
371	1	89.77	0.99	10.99	3.903043	23
292	1	86.82	0.99	10.99	4.823333	18
550	1	86.82	0.99	10.99	4.823333	18
297	1	86.8	0.99	10.99	4.34	20
372	2	85.87	0.99	10.99	6.605385	13
434	2	85.84	0.99	10.99	5.365	16
21	1	85.82	0.99	10.99	4.767778	18


200Lab
Education
Keyword
Takeaway
Section 3
APIRL - JULY
Social Media
Analysis

How do SQL Queries Active?




200Lab
Education
Keyword
Takeaway
Section 3
APIRL - JULY
Social Media
Analysis

How do SQL Queries Active?

